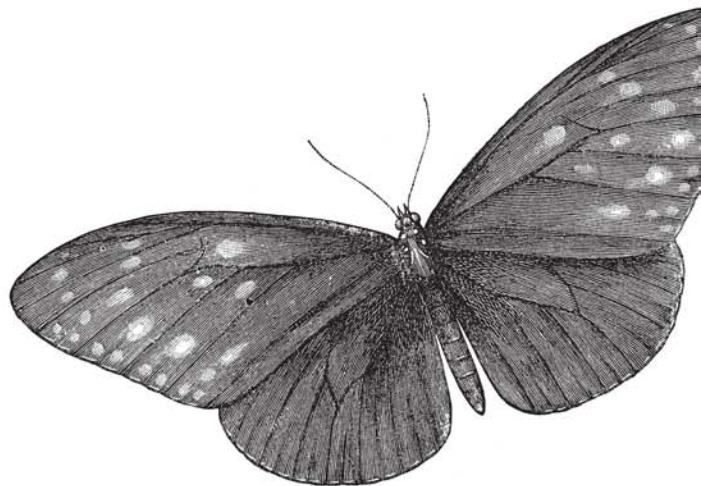


Get a Handle on Your Data



Learning

MySQL



O'REILLY®

*Seyed M.M. "Saied" Tabaghoghi
& Hugh E. Williams*

Learning MySQL



Whether you're running a business, keeping track of members and meetings for a club, or just trying to organize a large and diverse collection of information, you'll find the MySQL database engine to be a lifelong friend. And there's no better way to make MySQL work for you than *Learning MySQL*. This densely packed tutorial includes detailed instructions to help you set up and design an effective database, create powerful queries using SQL, configure MySQL for improved security, and glean useful information from your data.

After covering the basics, *Learning MySQL* travels far into MySQL's subtleties, including complex queries and joins, how to interact with the database over the Web using PHP or Perl, and important housekeeping such as backups and security.

Topics include:

- Installation on Linux, Windows, and Mac OS X
- Basic and advanced querying using SQL
- User management and security
- Backups and recovery
- Tuning for improved efficiency
- Command-line and web database applications using the PHP and Perl programming languages

MySQL is the most popular open source database, offering the power of a fully relational database in a package that's easy to set up and administer. Whether you've never touched a database or have already completed some MySQL projects, you'll find insights in *Learning MySQL* that will last a career.

Saied Tahaghoghi is a senior lecturer at the RMIT University School of Computer Science and Information Technology, and he is a member of the RMIT Search Engine Group, where he supervises research on text, image, video, and code retrieval.

Hugh E. Williams is a software design engineer at Microsoft's Windows Live Search in Redmond, Washington. He's published over 70 research papers and holds several patents, mostly in the search engine area.

www.oreilly.com

US \$44.99

CAN \$58.99

ISBN: 978-0-596-00864-2



9



Includes
**FREE 45-Day
Online Edition**

Learning MySQL

Learning MySQL

Seyed M.M. “Saied” Tahaghoghi and Hugh E. Williams

O'REILLY®

Beijing • Cambridge • Farnham • Köln • Sebastopol • Taipei • Tokyo

Learning MySQL

by Seyed M.M. “Saied” Tahaghoghi and Hugh E. Williams

Copyright © 2007 O’Reilly Media. All rights reserved.

Printed in the United States of America.

Published by O’Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O’Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://safari.oreilly.com>). For more information, contact our corporate/institutional sales department: (800) 998-9938 or corporate@oreilly.com.

Editor: Andy Oram

Production Editor: Sanders Kleinfeld

Copyeditor: Sanders Kleinfeld

Proofreader: Colleen Gorman

Indexer: Julie Hawks

Cover Designer: Karen Montgomery

Interior Designer: David Futato

Illustrators: Robert Romano and Jessamyn Read

Printing History:

November 2006: First Edition.

Nutshell Handbook, the Nutshell Handbook logo, and the O’Reilly logo are registered trademarks of O’Reilly Media, Inc. *Learning MySQL*, the image of blue spotted crows and related trade dress are trademarks of O’Reilly Media, Inc.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and O’Reilly Media, Inc. was aware of a trademark claim, the designations have been printed in caps or initial caps.

While every precaution has been taken in the preparation of this book, the publisher and authors assume no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.



This book uses RepKover™, a durable and flexible lay-flat binding.

ISBN: 978-0-596-00864-2

[M]

[01/09]

1259603998

Table of Contents

Preface	xi
----------------------	-----------

Part I. Introduction

1. Introduction	3
Why Is MySQL so Popular?	4
Elements of MySQL and Its Environment	5
MySQL Software Covered in This Book	7
The Book's Web Site	8
2. Installing MySQL	9
Installation Choices and Platforms	9
Using the Command-Line Interface	12
Using a Text Editor	18
Following the Instructions in This Book	20
Downloading and Verifying Files from the MySQL AB Web Site	22
Installing Under Linux	25
Installing Under Windows	51
Installing Under Mac OS X	61
Using a MySQL Installation Provided by an ISP	69
Upgrading an Existing MySQL Server	69
Configuring Access to the MySQL Server	74
What If Things Don't Work?	75
The Contents of the MySQL Directory	81
Configuring and Controlling the Apache Web Server	83
Setting up Perl	88
Resources	93
Exercises	94
3. Using the MySQL Monitor	95
Starting the Monitor	95

Style, Case, and Semicolons	97
The Monitor Help	98
Running the Monitor in Batch Mode	101
Loading the Sample Databases	102
MySQL Monitor Program Options	102
Graphical Clients	104
Exercises	105

Part II. Using MySQL

4. Modeling and Designing Databases	109
How Not to Develop a Database	109
The Database Design Process	111
The Entity Relationship Model	112
Entity Relationship Modeling Examples	120
Using the Entity Relationship Model	128
Using Tools for Database Design	130
Resources	132
Exercises	132
5. Basic SQL	135
Using the Music Database	135
The SELECT Statement and Basic Querying Techniques	139
The INSERT Statement	162
The DELETE Statement	168
The UPDATE Statement	171
Exploring Databases and Tables with SHOW and mysqlshow	173
Exercises	176
6. Working with Database Structures	179
Creating and Using Databases	179
Creating Tables	181
The Sample Music Database	212
Altering Structures	214
Deleting Structures	220
Exercises	221
7. Advanced Querying	223
Aliases	223
Aggregating Data	228
Advanced Joins	237
Nested Queries	250

User Variables	263
Transactions and Locking	266
Table Types	267
Exercises	275
8. Doing More with MySQL	277
Inserting Data Using Queries	277
Loading Data from Comma-Delimited Files	281
Writing Data into Comma-Delimited Files	284
Creating Tables with Queries	285
Updates and Deletes with Multiple Tables	288
Replacing Data	292
The EXPLAIN Statement	294
Exercises	295
9. Managing Users and Privileges	297
Understanding Users and Privileges	298
Creating and Using New Users	299
Privileges	303
The GRANT OPTION Privilege	306
How Privileges Interact	307
Users and Hosts	308
Checking Privileges	317
Revoking Privileges	323
Removing Users	324
Understanding and Changing Passwords	324
The Default Users	328
Devising a User Security Policy	333
Managing Privileges with SQL	339
Privileges and Performance	346
Resetting Forgotten MySQL Passwords	347
Exercises	349

Part III. Advanced Topics

10. Backups and Recovery	353
Dumping a Database as SQL Statements	353
Loading Data from an SQL Dump File	360
mysqlhotcopy	361
Scheduling Backups	361
The Binary Log	365
Checking and Repairing Corrupted Tables	366

Re-Creating Damaged Grant Tables	369
Resources	369
Exercises	369
11. Using an Options File	371
Configuring Options for the MySQL Monitor	371
Structure of the Options File	373
Scope of Options	374
Search Order for Options Files	375
Determining the Options in Effect	376
Exercises	377
12. Configuring and Tuning the Server	379
The MySQL Server Daemon	379
Server Variables	383
Checking Server Settings	388
Other Things to Consider	391
Resources	392
Exercises	392

Part IV. Web Database Applications with PHP

13. Web Database Applications	395
Building a Web Database Application	395
The Apache Web Server	400
Introducing PHP	402
Using a PHP-Enabled Web Hosting Site	410
Resources	411
Exercises	412
14. PHP	415
Language Basics	415
Accessing MySQL Using PHP	427
Modularizing Code	449
Processing and Using User Data	452
The PHP Predefined Superglobal Variables	455
Untainting User Data	456
Sessions	462
The Reload Problem	463
Using PHP for Command-Line Scripts	466
Resources	471
Exercises	471

15. A PHP Application: The Wedding Gift Registry	473
Designing and Creating the Wedding Database	474
The Login Form	476
Passing a Message to a Script	478
Logging Users In and Out	480
The db.php Include File	486
Editing the List of Gifts	487
Loading Sample Gifts	497
Listing Gifts for Selection	497
Selecting and Deselecting Gifts	501
Resources	507
Exercises	507

Part V. Interacting with MySQL Using Perl

16. Perl	511
Writing Your First Perl Program	512
Scripting With Perl	513
Resources	538
Exercises	538
17. Using Perl with MySQL	539
Connecting to the MySQL Server and Database	539
Handling Errors When Interacting with the Database	540
Using Queries That Return Answer Sets	542
Using Queries That Don't Return Answer Sets	545
Binding Queries and Variables	546
Importing and Exporting Data	552
Handling NULL Values	554
Resources	554
Exercises	554
18. Serving Perl Pages to the Web	557
The Perl CGI Module	560
Processing User Input	562
A Note on mod_perl	566
Perl Security	566
Resources	568
Exercises	568

Part VI. Appendix

Appendix: The Wedding Registry Code 571

Index 585

Preface

Database management systems are the electronic filing cabinets that help individuals and organizations to manage the mass of information they process each day. With a well-designed database, information can be easily stored, updated, accessed, and collated. For example, a freight company can use a database to record data associated with each shipment, such as the sender and recipient, origin and destination, dispatch and delivery time, current location, and shipping fee. Some of this information needs to be updated as the shipment progresses. The current status of a shipment can be read off the database at any time, and data on all shipments can also be summarized into regular reports.

The Web has inspired a new generation of database use. It's now very easy to develop and publish multi-user applications that don't require any custom software to be installed on each user's computer. Adding a database to a web application allows information to be automatically collected and used. For example, a customer can visit an online shopping site, see what's in stock, place an order, submit payment information, and track the order until the goods are delivered. He can also place advance orders for goods that aren't available, and submit reviews and participate in discussions on items he has purchased. If all goes well, the site's staff doesn't need to intervene in any of these actions; the less staff intervention required during normal operation, the more scalable the application is to large numbers of users. The staff are then free to do more productive tasks, such as monitoring sales and stock in real time, and designing special promotions based on product sales.

Both authors of this book have always been interested in using computers as a tool to make things faster, more efficient, and more effective. Over the past few years we've repeatedly found that the MySQL database management system—and the PHP and Perl programming languages—provide a perfect platform for serious applications such as managing research records and marking student assignments, and not-so-serious ones like running the office sweepstakes. On the way, we've learned a lot of lessons that we'd like to pass on; this book contains the tips that we think most readers will find useful on a daily basis.

Who This Book Is for

This book is primarily for people who don't know much about deploying and using an actual database-management system, or about developing applications that use a database. We provide a readable introduction to relational databases, the MySQL database management system, the Structured Query Language (SQL), and the PHP and Perl programming languages. We also cover some quite advanced material that will be of interest even to experienced database users. Readers with some exposure to these topics should be able to use this book to expand their repertoire and deepen their understanding of MySQL in particular, and database techniques in general.

What's in the Book

The book is divided into six main parts:

1. Introduction
2. Using MySQL
3. Advanced Topics
4. Web Database Applications with PHP
5. Interacting with MySQL using Perl
6. Appendix

Let's look at how the individual chapters are laid out.

Introduction

We first provide some context for the book in Chapter 1, where we describe how MySQL and web database applications fit into the domain of information management tools and technologies.

In Chapter 2, we explain how you can configure the software required for this book on different operating systems. This chapter provides far more detail than most books because we know that it's hard to learn MySQL if you can't first get it up and running.

Chapter 3 introduces the standard text-based interface to the MySQL server. Through this interface, you can control almost every aspect of the database server and the databases on it.

Using MySQL

Before we dive into creating and using databases, we look at proper database design in Chapter 4. You'll learn how to determine the features that your database must have, and how the information items in your database relate to each other.

In Chapter 5, we explore how to read data from an existing MySQL database and how to store data in it.

In Chapter 6, we explain how to create a new MySQL database and how to modify an existing one.

Chapter 7 covers more advanced operations such as using nested queries and using different MySQL database engines.

Chapter 8 continues the advanced operations theme; in this chapter, you'll find a discussion of importing and exporting data, and peeking under the hood to see how the MySQL server processes a given query.

In any serious application, you'll need to prevent unauthorized data access and manipulation. In Chapter 9, we look at how MySQL authenticates users and how you can allow or disallow access to data or database operations.

Advanced Topics

Data stored on a computer can be lost due to hardware failure, theft, or other incidents such as fire or flood. If you need your database, you'll save yourself a lot of hair-pulling by setting up regular and complete backups of your database structure and data. In Chapter 10, we introduce techniques that can help you easily recover from a data loss or corrupted database.

MySQL is highly configurable; in Chapter 11, we describe how you can use configuration files to modify the behavior of the MySQL server and associated programs.

In Chapter 12, we introduce several ways to customize your MySQL server and your application database for improved performance. Small speedups for frequently used queries can markedly improve the overall performance of your system.

Web Database Applications with PHP

In Chapter 13, we examine how web database applications work.

Chapter 14 follows with an introduction to the PHP programming language and a discussion of how PHP can be used to access and manipulate data in a MySQL database.

In Chapter 15, we walk through the design of a wedding gift registry to illustrate the process of developing a full-fledged web database application.

Interacting with MySQL Using Perl

In Chapter 16, we present an easy-to-follow introduction to the powerful Perl programming language.

We continue in Chapter 17 by using the Perl DBI module to connect to a MySQL database to store and read information, and to import and export data.

We conclude this part in Chapter 18 by using the Perl CGI module to create dynamic web pages that can interact with a MySQL database.

Appendix

The Appendix contains all the source code for the wedding gift registry developed in Chapter 15. You can download this source code, and much more, from the book's web site.

Conventions Used in This Book

This book uses the following typographical conventions:

Italic

Indicates nomenclature that we've not previously used. Also used for emphasis and to indicate files and directories.

Constant Width

Indicates commands and command options, usernames, and hostnames. Also used to show the command output, and the contents of text and program files.

Constant Width Bold

Used in examples to indicate commands or other text that should be typed literally by the user.

Constant Width Italic

Indicates text that you should replace with your own values—for example, your own name or password. When this appears as part of text that you should type in, it is shown as ***Constant Width Italic Bold***.

#, \$

Used in some examples as the root shell prompt (**#**) and as the user prompt (**\$**) under the Bourne or **bash** shell. Unless stated otherwise, instructions in such examples can be used with little modification from the Windows command prompt.

C:\>

Used in some examples as the Windows command prompt.



Signifies a tip, suggestion, or general note.



Indicates a warning or caution.

Resources

Each chapter finishes with a list of books and web sites that contain further information on the topics covered. The book also has a companion web site at <http://www.learningmysql.com> that contains links to useful resources, frequently asked questions (FAQs), and the example code and data used in this book. It's probably a good idea to have a quick look at the web site now so that you know what's there; it could save you a lot of searching and typing!

Using Code Examples

This book is here to help you get your job done. In general, you may use the code in this book in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing a CD-ROM of examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Learning MySQL* by Seyed M.M. Tahaghoghi and Hugh E. Williams. Copyright 2007 O'Reilly Media, Inc., 978-0-596-00864-2."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

Safari® Enabled



When you see a Safari® Enabled icon on the cover of your favorite technology book, that means the book is available online through the O'Reilly Network Safari Bookshelf.

Safari offers a solution that's better than e-books. It's a virtual library that lets you easily search thousands of top tech books, cut and paste code samples, download chapters, and find quick answers when you need the most accurate, current information. Try it for free at <http://safari.oreilly.com>. (<http://safari.oreilly.com>)

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North

Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international/local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, or any additional information. You can access this page at:

<http://www.oreilly.com/catalog/learnmysql>

To comment or ask technical questions about this book, send email to:

bookquestions@oreilly.com

For more information about books, conferences, Resource Centers, and the O'Reilly Network, see the O'Reilly web site at:

<http://www.oreilly.com>

We've spent a lot of effort trying to ensure that the material in this book is correct and that the instructions and examples will work in your environment. However, there is always room for improvement, and we're keen to know your thoughts on how we can make things better. Please send your thoughts by email to *saied@tahaghoghi.com*, or use the online feedback form at *<http://www.learningmysql.com/feedback>*.

Acknowledgments

First of all, we thank all the people around the world who have contributed to the MySQL, PHP, and Perl projects, and related open source initiatives such as Linux. These have been an important part of our lives, and of course, this wouldn't be much of a book without them!

Writing a technical book is an incredibly time-consuming process, and only an editor as patient and steady-handed as Andy Oram could have coaxed and cajoled us into getting the book done and out of the door. Thanks Andy!

We also thank our technical reviewers, Paul Kinzelman, Falk Scholer, and Omkhar Arasaratnam, for pointing out many ways in which the content could be improved, and the team at O'Reilly for converting our material into a professionally produced book.

Saied Tahaghoghi

I thank Hugh for inviting me to collaborate on this project (and for the countless other ways he's made my life more interesting); Santha Sumanasekara for helping me to set up my first ever Linux box and introducing me to MySQL and PHP so many years ago; my various teachers and mentors for painstakingly showing me the way; and my friends and colleagues for helping me maintain an appearance of sanity. Most of all, I thank all my family for their constant kindness, support, and prayers; I'm especially indebted

to my wife, Somayyeh, for patiently enduring for so long my claims that, “The book’s almost done!”

Hugh Williams

I thank Selina Williams for being always patient, even-tempered, encouraging, and ready to listen while I slaved away on yet another (and maybe my last?) book project. Thanks also to Lucy and Rose for letting Dad work upstairs day after day, and to Mum and Dad for the lend of the Winnebago in the paddock while I bashed out a few of the more technical chapters. But most of all, thanks Saied for agreeing to take up the reins and finish the book after I moved to Microsoft: you’re one of the best men I know. Last, another thank you to Andy Oram; you’re a very patient guy whom I’ve learnt a lot from.

PART I

Introduction

Introduction

MySQL (pronounced “My Ess Cue Ell”) is more than just “the world’s most popular open source database,” as the developers at the MySQL AB corporation (<http://www.mysql.com>) claim. This modest-sized database has introduced millions of everyday computer users and amateur researchers to the world of powerful information systems.

MySQL is a relatively recent entrant into the well-established area of *relational database management systems* (RDBMs), a concept invented by IBM researcher Edgar Frank Codd in 1970. Despite the arrival of newer types of data repositories over the past 35 years, relational databases remain the workhorses of the information world. They permit users to represent sophisticated relationships between items of data and to calculate these relationships with the speed needed to make decisions in modern organizations. It’s impressive how you can go from design to implementation in just a few hours, and how easily you can develop web applications to access terabytes of data and serve thousands of web users per second.

Whether you’re offering products on a web site, conducting a scientific survey, or simply trying to provide useful data to your classroom, bike club, or religious organization, MySQL gets you started quickly and lets you scale up your services comfortably over time. Its ease of installation and use led media analyst Clay Shirky to credit MySQL with driving a whole new type of information system he calls “situated software”—custom software that can be easily designed and built for niche applications.

In this book, we provide detailed instructions to help you set up MySQL and related software. We’ll teach you Structured Query Language (SQL), which is used to insert, retrieve, and manipulate data. We’ll also provide a tutorial on database design, explain how to configure MySQL for improved security, and offer you advanced hints on getting even more out of your data. In the last five chapters, we show how to interact with the database using the PHP and Perl programming languages, and how to allow interaction with your data over the medium most people prefer these days: the Web.

Why Is MySQL so Popular?

The MySQL development process focuses on offering a very efficient implementation of the features most people need. This means that MySQL still has fewer features than its chief open source competitor, PostgreSQL, or the commercial database engines. Nevertheless, the skills you get from this book will serve you well on any platform.

Many database management systems—even open source ones—preceded MySQL. Why has MySQL been the choice for so many beginners and small sites, and now for some heavyweight database users in government and industry? We can suggest a few factors:

Size and speed

MySQL can run on very modest hardware and puts very little strain on system resources; many small users serve up information to their organizations by running MySQL on modest desktop systems. The speed with which it can retrieve information has made it a longstanding favorite of web administrators.

Over the past few years, MySQL AB has addressed the need of larger sites by adding features that necessarily slow down retrieval, but its modular design lets you ignore the advanced features and maintain the suppleness and speed for which MySQL is famous.

Ease of installation

Partly because MySQL is small and fast, it works the way most people want straight “out of the box.” It can be installed without a lot of difficult and sophisticated configuration. Now that many Linux distributions include MySQL, installation can be almost automatic.

This doesn’t mean MySQL is free of administrative tasks. In particular, we’ll cover a few things you need to do at the start to tighten security. Very little configuration is shown in this book, however, which is a tribute to the database engine’s convenience and natural qualities.

Attention to standards

As we’ll explain in the “Structured Query Language” section later in this chapter, multiple standards exist in the relational database world, and it’s impossible to claim total conformance. But learning MySQL certainly prepares you for moving to other database engines. Moving code from one database engine to another is never trivial, but MySQL does a reasonable job of providing a standard environment, and gets better as it develops more features.

Responsiveness to community

With a few hundred employees scattered around the globe, MySQL AB is a very flexible organization that keeps constant tabs on user needs. At its conferences, lead developers get out in front and make themselves available to everyone with a gripe or a new idea. There are also local MySQL user groups in almost every major city. This responsiveness is helped by the fact that MySQL is open and free; any

sufficiently skilled programmer can look at the program code to find and perhaps help in fixing problems.

MySQL actually has a dual-license approach: if you want to build your own product around it, you pay MySQL AB a license fee. If you just want to use MySQL to serve your own data, you don't have to pay the license fee. MySQL also offers technical support, as do numerous other companies and consultants, some of them probably near you.

Easy interface to other software

It is easy to use MySQL as part of a larger software system. For example, you can write programs that can interact directly with a MySQL database. Most major programming languages have libraries of functions for use with MySQL; these include C, PHP, Perl, Python, Ruby, and the Microsoft .NET languages. MySQL also supports the Open Database Connectivity (ODBC) standard, making it accessible even when MySQL-specific functionality isn't available.

Elements of MySQL and Its Environment

You need to master several skills to run a database system. In this section, we'll lay out what goes into using MySQL and how we meet those needs in this book.

A MySQL installation has two components: a *server* that manages the data, and *clients* that ask the server to do things with the data, such as change entries or provide reports. The client that you'll probably use most often is the `mysql` “MySQL monitor” program, provided by the MySQL AB company and available in most MySQL installations. This allows you to connect to a MySQL server and run SQL queries. Other simple clients are included in a typical installation; for example, the `mysqladmin` program is a client that allows you to perform various server administration tasks.

In fact, any program that knows how to talk to the MySQL server is a client; a program for a web-based shopping site or an application to generate sales graphs for a marketing team can both be clients. In Chapter 3, you'll learn to use the MySQL monitor client to access the MySQL server. In Chapters 13 through 15, we'll look at how we can use PHP to write our own custom clients that run on a web server to present a web frontend to the database for this. We'll use the Apache web server (<http://httpd.apache.org>). Apache has a long history of reliable service and has been the most popular web server in the world for over 10 years. The Apache web server—or “HTTP server”—project is managed by the Apache Foundation (<http://www.apache.org>). Although the web server and MySQL server are separate programs and can run on separate computers, it's common to find small- to medium-scale implementations that have both running on a single computer. In Chapters 16 through 18, we'll explore how the Perl programming language can be used to build command-line and web interfaces to the MySQL server.

To follow the content in this book, you will need some software; fortunately, all the software we use is open source, free for noncommercial use, and easily downloaded from the Internet. To cover all parts of this book, you need a MySQL database server, Perl, and a web server that can talk to MySQL using the PHP and Perl programming languages. We'll explore four aspects of using MySQL:

MySQL server

We explain how to create your own MySQL installation, and how to configure and administer it.

SQL

This is the core of MySQL use, and the major topic in this book. It's introduced in "Structured Query Language."

Programming languages

SQL is not a simple or intuitive language, and it can be tedious to repeatedly perform complex operations. You can instead use a general-purpose programming language such as PHP or Perl to automatically create and execute SQL queries on the MySQL server. You can also hide the details of the interaction with the database behind a user-friendly interface. We'll show you how to do this.

Web database applications

We explain how you can use PHP or Perl to create dynamic, database-driven web applications that can publish information from the database to the Web, and capture information provided by users.

HTML is the lingua franca of the Web. Although learning HTML is not within the scope of this book, there are many great HTML guides available, including *HTML and XHTML: The Definitive Guide* by Chuck Musciano (O'Reilly). We recommend that you pick up the basics of HTML before reading Chapters 13, 14, 15, or 18.

The LAMP Platform

It's very common to find web database applications developed using the Linux operating system, the Apache web server, the MySQL database management system, and the Perl or PHP scripting language. This combination is often referred to by the acronym LAMP, a term invented at O'Reilly Media.

Linux is the most common development and deployment platform, but as we'll show in this book, you can run all the tools on other operating systems. In fact, we'll give directions for getting started on Linux, Windows, and Mac OS X. Most of the content in this book can be used for other operating systems with little modification.

The P in LAMP originally stood for Perl, but over the past decade, users have increasingly turned to PHP for developing dynamic web pages. PHP is very clean and efficient for retrieving data and displaying it with minimal processing. If you have to do heavy data crunching after the data is returned from MySQL, Perl may still be a better choice. We discuss PHP and Perl largely independently; you can pick up one without needing

to learn the other, although we believe that you'll benefit from learning both languages. In fact, almost any modern language can be used to perform this task; most of them have the necessary interfaces to both web servers and database engines.

Structured Query Language

IBM is to be credited not only with inventing the relational database, but developing the language still used today to interact with such databases. SQL is a little odd, bearing the stylistic marks of its time and its developers. It's also gotten rather bloated over the years—a process made worse by its being standardized (multiple times)—but in this book we'll show you the essentials you really need and help you become fluent in them.

SQL shows many of the problems that are commonly attributed to computing standards: it tries to accomplish too much, it forces new features into old molds to maintain backward compatibility, and it reflects uneasy compromises and trade-offs among powerful vendors. As a result, there are several standards that database management systems can adhere to. SQL-92 dates back to 1992 and provides just about everything that you will need for beginning work. However, it lacks features demanded by some modern applications. SQL:1999 was standardized in 1999 and adds a huge number of new features, many of them considered overkill by some experts. There is also a more recent standard, SQL:2003, that was published in 2003 and adds support for XML data.

Each development team has to decide on the trade-offs between the features requested by users and the need to keep software fast and robust, and so database engines generally don't conform totally to any one standard. Furthermore, historical differences have stayed around in legacy database engines. That means that even if you use fairly simple, vanilla SQL, you may have to spend time when porting your skills and your code to another database engine.

In this book, we'll show you how to use MySQL's flavor of SQL to create databases and store and modify data. We'll also show you how to use this SQL variant to administer the MySQL server and its users.

MySQL Software Covered in This Book

You can be very productive with MySQL without dedicating a lot of time to configuration and administration. In Chapter 2, we'll look at several common ways of setting up the software you'll need for this book. While you can skip most of the instructions if you already have a working MySQL installation, we recommend you at least skim through the material for your operating system; we'll frequently refer to parts of this chapter later on. As part of this chapter, we explain how you can configure your MySQL server for good security.

MySQL provides many other tools for administration, including compile-time options, a large configuration file, and standalone utilities developed by both MySQL AB and

external developers. We'll give you the basics that will keep you up and running in most environments, and will briefly describe even some relatively advanced topics.

We won't cover all the programs that come with the MySQL distribution, and we won't spend too long on each one; the MySQL reference manual does a good job of covering all the options. We'll instead look at the programs and options that you're most likely to use in practice; these are the ones we've used ourselves a reasonable number of times over several years of working with MySQL.

The Book's Web Site

We've set up the web site, *<http://www.learningmysql.com>*, which contains the sample databases, datafiles, and program code. We recommend you make good use of the web site while you read this book.

Installing MySQL

Learning MySQL is easiest if you have a database server installed on your computer. By administering your own server, you can go beyond querying and learn how to manage users and privileges, configure the server, and make the best use of its features. Importantly, you also learn the steps required to install and configure MySQL, which is useful when you need to deploy your applications elsewhere.

This chapter explains how to choose and configure a suitable environment for learning MySQL. We cover the following topics:

- What to install: how to decide between precompiled packages, an integrated web development environment, and compiling from the source code
- Where to install: Linux, Microsoft Windows, or Mac OS X?
- Why, when, and how to upgrade MySQL
- How MySQL has changed and how to migrate between versions
- How to configure the Apache web server and support for the PHP and Perl scripting languages.

MySQL is available in several forms and for many operating systems. In the next section, we examine the choices available and how you can decide what suits you.

Installation Choices and Platforms

As we mentioned before, you'll need MySQL, the Apache web server, PHP, and Perl for this book. How you choose to install these depends on what you want to do, how confident you are in using your operating system environment, and the level of privileges you have on your system. If you're planning to use the installation for learning and development only, and not for a production site, then you have greater choice, and you need not be so concerned about security and performance. We'll describe the most common ways to install the software you need.

You can find the ready-to-use MySQL programs (known as *binaries*) on the MySQL AB web site and on Linux installation CDs and web sites. You can also download the

source code for MySQL from the MySQL AB web site and prepare, or *compile*, the executable programs yourself. By doing the compiling yourself, you ensure that you have the most up-to-date version of the software, and you can optimize the compiler output for your particular needs. The MySQL manual says that you can get a performance increase of up to 30 percent if you compile the code with the ideal settings for your environment. However, rolling your own installation from source code can also be a tedious and error-prone process, so we suggest that you stick with the ready-made binaries unless you're experienced and really need to squeeze every ounce of performance from your server. Compiling from source under Windows and Mac OS X is even more involved, so it's uncommon, and we don't discuss it in this book.

You can also install MySQL as part of an integrated package that includes the Apache, PHP, and Perl software that you'll need later in this book. Using an integrated package allows you to follow a step-by-step installation wizard. It's easier than integrating standalone packages, and many of the integrated packages include other tools that help you adjust configuration files, work with MySQL, or conveniently start and stop services. Unfortunately, many of the integrated packages are a couple of minor releases behind the current version and may not include all the PHP libraries that you require. Another disadvantage is that an integrated package doesn't always fit in with your current setup; for example, even if you already have a MySQL installation, you'll get another one as part of the integrated package, and you'll have to take care to avoid clashes. Despite the disadvantages, we recommend you follow this approach. There are several integrated packages available; we feel that XAMPP is probably the best produced of these, and we'll describe how to install and use this. XAMPP includes MySQL, the Apache web server with PHP and Perl support, and other useful software such as phpMyAdmin. We recommend that you start out by using XAMPP, and we won't spend time describing how to separately install and configure Apache, PHP, and Perl to work together on your system.

The software packages you need—MySQL, Apache, PHP, and Perl—are available ready to install on many operating systems and can be compiled to run on a large number of others. However, chances are that you're running one of three major operating systems: Linux, Windows, or Mac OS X, so we'll provide detailed instructions for only these three. Let's see how they compare as MySQL development and production platforms.

Linux

Linux is an open source operating system that is closely modeled on Unix, which is why it's often called a Unix clone. Even though it's free, Linux is very powerful and very secure, with versions available for a wide range of hardware.

You typically get Linux in the form of a *distribution*, such as Red Hat or Mandriva. A distribution packages the operating system together with a large range of useful software for things such as word processing, networking, web and database development,

and even games. These distributions are free to download and distribute; you can also buy low-cost CD copies or more expensive shrink-wrapped packs with printed manuals. Many of the most popular web sites run on Linux, and it's an excellent choice for learning MySQL.

Live CDs

You can install Linux on its own, or alongside Windows on a single computer (this is known as *dual-booting*). If you want to try out Linux without installing it on your computer, you can use a bootable, or *live*, CD distribution. This allows you to boot your computer from a CD to get a fully-working Linux system without making any changes to your hard disk. When you remove a live CD and reboot, everything is back to what you had before; you don't have to worry about doing any damage while you learn how to use Linux. For example, the Knoppix (<http://www.knoppix.org>) live CD includes all the software—MySQL, the Apache web server, PHP, and Perl—that you need for this book. However, we recommend that you use a live CD only to become familiar with Linux. While it's possible to save files from a live CD onto the hard disk, a USB flash disk, or another computer through a network connection, this is tedious. For anything that involves using Linux for extended periods of time, you're better off with a full installation to hard disk.

Windows

Microsoft Windows is by far the most common commercial PC operating system today, and new PCs often come with Windows pre-installed. Windows XP, released in 2001, is available on most current PCs. Windows Vista is the latest version of Windows; at the time of writing, it's in "release candidate" (for testing) form and due to be published in the next few months.

We've tested the instructions in this book using both XP and Vista. While we wouldn't recommend using either version for a production server, they're quite appropriate for learning the material in this book. We assume you're using either XP or Vista; you can set up a suitable environment on older versions of Windows such as 98 and Me, but the process is less straightforward. When we say "Windows" in this book, we mean XP or Vista.

Mac OS X

All new Apple computers since 2001 have come with OS X; recent versions include 10.3 (Panther) and 10.4 (Tiger), with 10.5 (Leopard) due for release in the next few months. OS X has a nice graphical user interface over a Unix-like heart, which means it's not hard to use software originally designed for Unix or Linux. Most new Apple computers built from 2006 onward have an x86-type processor; older systems have a PowerPC processor. You can easily check which operating system version or processor your system has by clicking on the Apple menu and choosing the About This Mac entry.

It's not common to find a production MySQL server running on OS X, but it's a good environment for learning MySQL.

So, What Should I Do?

As we mentioned earlier, you can use almost any major operating system when practicing the material covered in this book, but to keep things sensible, we'll assume you're using one of the big three just listed. Where the process varies between operating systems, we'll clearly explain the necessary steps. It shouldn't be too hard to interpret the instructions for other operating systems that we don't focus on in this book. For example, many of the Linux instructions can be used with little adaptation on Solaris or FreeBSD.

To install a MySQL server with the standard directories and settings for a system-wide installation, you'll generally need *superuser* (also known as the *system root user* or *administrator*) privileges on your system. Always be careful when using superuser access. The superuser can do anything on a system, so you might be tempted to always log in under the superuser account. However, "anything" means *anything*—including accidentally deleting vital system files and making the system unusable. There are also security risks associated with using this level of access by default, so we strongly suggest you stick to an ordinary, or *nonprivileged*, user account and switch to the privileged account only when necessary. We'll explain how to configure a MySQL server installed on a Linux or Mac OS X system to run as a less privileged user; any files and directories that the server creates are then owned by this account.

If you don't have superuser access—for example, you're using a shared university computer or want to experiment without touching the system-wide MySQL installation—you can generally install a local MySQL server using nonstandard settings; we'll also explain how you can do this. However, we recommend that you go with the default settings if you can, at least while you're still learning a lot about MySQL. You're far less likely to make mistakes, and less likely to run into difficulties with the software; programs are rarely tested as well on nonstandard configurations as they are on the default settings.

Finally, there are cases when the database server may already be set up for you. Many hosting companies, for example, allow you to administer your databases using only a web-based MySQL client such as phpMyAdmin. We'll take a brief look at phpMyAdmin in Chapter 13.

Using the Command-Line Interface

The three operating systems we use in this book all have graphical user interfaces; you can start programs by clicking on icons, you can select tasks from menus, and you can drag and drop files and folders. However, once you start to use more powerful aspects of the operating system and applications, you'll quickly find that some tasks are more

easily done by typing in commands. For example, you can tell the operating system to list certain files in a folder or run a given program in a particular way.

Linux, Windows, and Mac OS X all have a command-line interface that allows you to do this. In Linux and Mac OS X, you use a Terminal program to show you the command-line interface, which is called the *shell*. In Windows, you use the Command Prompt Window program to show you the Command Prompt, sometimes called the DOS prompt.

In this section, we'll describe how each command-line interface works; you can skip the descriptions for the operating systems you don't use.

The Linux and Mac OS X Shell

To access the shell under Linux, open a terminal program, such as `konsole`, `rxvt`, or `xterm`; these are often listed in the main menu under the System or System Tools group, and may be simply labeled Terminal. To access the shell under Mac OS X, open a terminal window by double-clicking on the Terminal icon in the Utilities folder under the Applications group.

Under Linux, you'll see a prompt similar to this one:

```
[adam@eden ~]$
```

while under Mac OS X, you'll see something like this:

```
eden:~ adam$
```

This *shell prompt* indicates what user account you're logged in under, what computer you're logged in to, and what directory you're working from. You'll generally be first logged in as an ordinary user (we've shown the user `adam` here) on the computer (`eden`), and working from your home directory. The tilde (`~`) character is a shortcut symbol to a user's home directory on any Unix-like system, including Linux and Mac OS X; for example, a user's home directory could be `/home/adam`, but you can refer to it as `~adam`, or, if you're logged in as `adam`, simply as `~`. The sample prompt shows that the user `adam` is logged in to the computer `eden` and working from his home directory. To keep things simple, we'll just show a dollar sign to indicate the Linux or Mac OS X shell prompt, as below:

```
$
```

From the shell, you can run many useful commands; we'll see some as we progress through this book. Two standard commands that are important to know for this book are:

`cd`

Changes your working folder or directory on disk. For example, you can change to the `/tmp` directory by typing:

```
$ cd /tmp
```

You can also change to your home directory by using the tilde shortcut:

```
$ cd ~
```

In fact, you can leave out the tilde: `cd` on its own means “change to my home directory.”

`ls`

Lists the files and directories in your working folder. For example, you can list the files in your home directory by typing:

```
$ ls ~
```

Together, the `cd` and `ls` commands are the text equivalent of using a graphical file manager—such as Konqueror or Nautilus under Linux, or the Finder under Mac OS X—to go to different directories and view their contents.

Command completion and history

Command completion is a great time-saving feature; when you start to type the name of a command, file, or directory, pressing the Tab key cycles through names that could match. The best way to understand this is to try it. For example, when you type:

```
$ cd /t
```

and then repeatedly press the Tab key, you’ll see items beginning with the letter “t” in the / (filesystem root) directory. If a name has spaces, a backslash character is added automatically before each space—for example `My\ Important\ Notes.txt`. Most Linux and Mac OS X systems are configured to use the `bash` shell, and we assume you’re using this, too. If you’re using a different shell variant, such as `tcsh`, you’ll need to press the Ctrl-D key combination in place of the Tab key.

Pressing the up and down arrow keys will cycle through the last commands you typed; you can use the arrow keys to edit a previous command, and you can press the Enter key to run a displayed command. You can see a list of recently used commands with the `history` command, as below:

```
$ history
1 cd Photos/
2 lt
3 find . -name "*AMES*"
4 cfdisk /dev/hda
5 ssh ubuntu@192.168.1.1
```

You can quickly run a command again by typing the number preceded by an exclamation mark (!) character. For example, to run the command numbered 3 in the history list, you can type `!3` and press the Enter key.

Performing restricted operations

Certain restricted operations on a Linux or Mac OS X system are allowed only if you have *superuser*, or *root*, privileges. On a Linux system, you can log in as the system root

user by typing the `su -` (switch user) command. When prompted, type in the system root user's password and press the Enter key:

```
[adam@eden ~]$ su -  
Password: the_system_root_password  
[root@eden ~]#
```

This is almost identical to the case for Mac OS X:

```
eden:~ adam$ su -  
Password: the_system_root_password  
eden:~ root#
```

After you type in the password, you'll be logged in as the user `root` on the same computer (in this example, `eden`) and be working from that user's home directory (also indicated by a tilde).

Notice how the last character of the prompt is a dollar sign (\$) when you're not the root user and the hash or pound (#) sign when you are. In this book, we'll use these symbols to indicate whether you should run a certain command as an ordinary user or as the root user. When you've finished doing the restricted operations, you can log out from the system root account by typing **exit**:

```
# exit  
$
```

You can generally use the `sudo` command to perform actions with system superuser privileges, even though you're not actually logged in as `root`. You can also use the `sudo -s` command to log in as the root user (in place of `su -`). If you log in as the system root user, you can then omit the `sudo` keyword. Again, we emphasize that you can inadvertently do a great deal of damage if you use the root account, and we recommend that you log in as the system root user as infrequently as you can. Some configuration is necessary to allow ordinary Linux users to use the `sudo` command, but it's enabled by default under Mac OS X, and we'll use this approach when discussing installation for this operating system.

You can add the ampersand symbol (&) at the end of a command to start the command in the background, allowing you to use the shell for other work. It's better to avoid using this symbol in conjunction with the `sudo` command, since you won't see any system prompt for you to enter your password. When we want you to run a `sudo` job in the background, we'll ask you to start the job normally, then press the CTRL-Z key combination to suspend this new job. You can then type the command **bg** to send the suspended job to the background.

Restricting access to files and directories

Before we end our discussion of the Linux and Mac OS X shell, let's look at how access to files and directories is controlled under such Unix-like operating systems. Each file or directory can have read, write, and execute permissions set for the *user* who owns it, the *group* associated with it, and *every other user*.

When the operating system is asked to allow access to a file or directory, it looks to see who the user is and what groups this user belongs to. It then checks the user and the group associated with that file or directory, and allows access only if the permission settings are appropriate.

Your group on a Linux or Mac OS X system is typically the same as your username, so, for example, the username and group for the user `adam` would both be `adam`. The user and group associated with a file or directory can be changed by using the `chown` command and specifying the username and group as `username:group`. For example, you can set the owner of `myfile.txt` to be `adam`, and the associated group to be `managers`, by typing:

```
# chown adam:managers myfile.txt
```

Only the superuser is allowed to change the owner of a file or directory.

You can allocate permissions to a file or directory by using the `chmod` command. To allow the user who owns the file `myfile.txt` to read and write (modify) it but allow other users to only read it, you would write:

```
$ chmod u=rw,g=r,o=r myfile.txt
```

You can also ensure that only the user who owns the file can read and write to the file as follows:

```
$ chmod u=rw,g=,o= myfile.txt
```

Here, the group and other users have been assigned no permissions. Similarly, you can give everyone read, write, and execute permissions to the directory `mydir` by typing the command:

```
$ chmod u=rwx,g=rwx,o=rwx mydir
```

When reading other documentation, you'll probably also come across cases where an octal value (or *mask*) is used with the `chmod` command. In this notation, read access has the value 4, write access has the value 2, and execute access has the value 1. So, read-only access has the value 4, but read-and-write access has the value 4+2=6. Our previous two examples would be written as:

```
$ chmod 644 myfile.txt
```

and:

```
$ chmod 777 mydir
```

The `chown` or `chmod` operation can be applied to all files and directories under a specified directory by using the `--recursive` option (under Linux) or the `-R` option (under Mac OS X as well as Linux). We'll see examples of this later in this chapter.

The Windows Command Prompt

Under Windows, you can open a command-prompt window by clicking on the Command Prompt entry under the Accessories submenu. You can also type `cmd` in the Start menu search box (Vista) or in the Start menu “Run...” field (XP).

The command prompt typically shows you the current working disk and directory:

```
C:\Documents and Settings\Adam>
```

In this example, the current working directory is the home directory `\Documents and Settings\Adam` on the C: disk. Under Vista, the location of the home directory is slightly different:

```
C:\Users\Adam>
```

From the command prompt, you can run many useful commands; we’ll see some as we progress through this book. Two standard commands that are important to know for this book are:

cd

Changes your working folder or directory on disk.

dir

Lists the files and directories in your working folder.

Together, the `cd` and `dir` commands are the text equivalent of using a graphical file manager such as Windows Explorer to go to different directories and view their contents.

Windows uses the variable `%HOMEPATH%` to refer to your home directory, so you can always change to your home directory by typing:

```
C:\> cd %HOMEPATH%  
C:\Documents and Settings\Adam>
```

Command completion and history

Command completion is a feature that can save you a lot of typing. When you start to type the name of a command, file, or directory, pressing the completion key sequence cycles through matches. The completion key varies between systems; it is generally the Tab key or the Ctrl-D or Ctrl-F key combination.

Under Windows, you can activate the command-completion feature if you start the command prompt with the `/f:on` option (command completion is active by default in Vista). If the `/f:on` switch doesn’t work on your system, try calling the `cmd` program without the switch. You can also configure Windows XP to have command completion active by default, but we won’t describe how to do this here.

The best way to understand command completion is to try it out. For example, when you type `cd c:\p:`

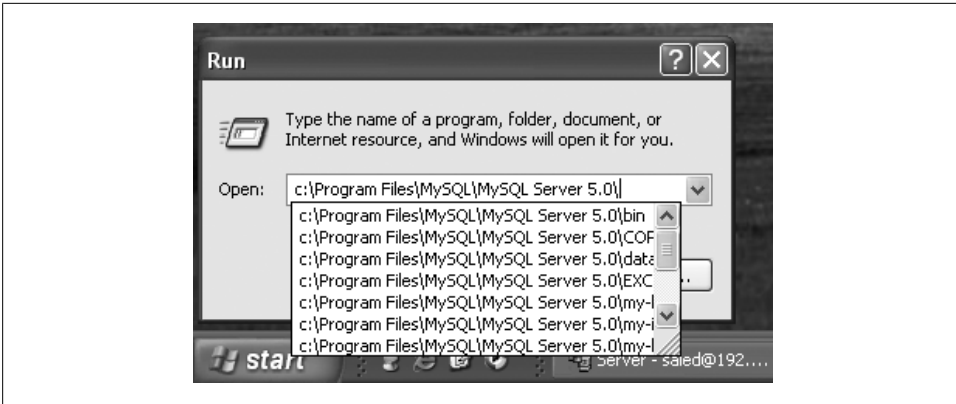


Figure 2-1. Starting a program from the Run menu item

```
C:\> cd c:\p
```

and then repeatedly press the completion key sequence, you'll see items beginning with the letter "p" in the C:\ directory. Note that Windows doesn't mind whether you use uppercase or lowercase when referring to files and folders.

Quotes are added automatically around names with spaces—for example, "C:\Program Files". To continue expansion, press the backspace key to delete the last quote and type a further hint. For example, to switch to the C:\Program Files\MySQL directory, you'd delete the quote, type a backslash (\), and then press the completion key sequence again.

Pressing the up and down arrow keys will cycle through the command history. You can see a list of recently used commands with the `doskey/history` command, as below:

```
C:\> doskey/history
dir C:\
doskey/history
```

There are many more tweaks for the command prompt; just do a search on the Web for "windows cmd".

You can also start other programs from the Start menu; under XP, you would use the Run menu item to browse to select the program you want. If you type in the command, you'll also get command completion, as shown in Figure 2-1. Under Vista, simply type the name of the program in the Start menu search box. However, this approach doesn't always keep the results of running a program on the screen, so we suggest you use the command-prompt window.

Using a Text Editor

As you read through this book, you'll frequently find references to using a text editor. This means a program that can edit and save files that contain only plain text. Word

processors save additional formatting instructions that only other word processors understand. Word processing programs also tend to use proportional fonts, which makes it hard to read and write files of scripts and commands. It *is* possible to use a word processor to load and save plain-text files, but it's rather inconvenient and error-prone, and so we don't recommend you do this.

So, what should you use? There are hundreds of text editors available, and most people find one they prefer to use. You should try out several different programs and settle on one that you're comfortable with. Let's look at some options:

Linux

Under Linux, popular text editors include `pico`, `gvim`, `vim`, `emacs`, `joe`, `kate`, `gedit`, and `xedit`. You can often find these listed under the Editors group in the main menu of most Linux distributions. If you're curious, you can also type the command `apropos "text editor"` at the shell to see a list of programs that have the phrase "text editor" in their description.

Windows

Under Windows, use Notepad; you can also download and install free text editors such as `gvim`, or commercial editors such as `EditPad` and `TextPad`.

Mac OS X

Under Mac OS X, you can use the included editors `pico`, `vim`, or `emacs`, configure the TextEdit program to behave as a text editor, or install and use other editors such as BBEdit or Smultron.

To start an editor from the command line, type in the name of the program followed by the name of the file you wish to edit; for example, you can open the file *myfile.txt* with the `pico` editor by typing:

```
$ pico myfile.txt
```

You can also open files from the graphical user interface; double-clicking on the text-file icon will generally open the file in a text editor. You can modify the program that's used to open text files by right-clicking on the text-file icon (in Windows, depress the Shift button while clicking) and work your way through the program options. We won't go into detail here.

Under Mac OS X, you can also configure the TextEdit program to act as a text editor. Start the TextEdit program, and then choose the Preferences option from the TextEdit menu. In the dialog box that appears, select Plain Text under the Format heading. To open a file with TextEdit from the command line, you should type:

```
$ open -a TextEdit myfile.txt
```

You can instead select the plain-text mode for individual files one at a time by selecting the Make Plain Text option from the Format menu, but this approach is likely to be tedious and error-prone over time.

Following the Instructions in This Book

Starting in the next section, we'll explain how to configure a MySQL server on the same system that you're logged in to (that is, `localhost`). We won't describe how to set up the MySQL server on one computer and the web server on a different computer; it shouldn't be too hard to modify our instructions to do this. If you modify any of the default settings, you'll need to remember to specify them where necessary.

We also assume that if you're using Windows, you use only the `C:` disk; we'll explain how and when to change your working directory. When we show only the Linux or Mac OS X prompt as below:

```
$
```

or the Windows Command Prompt as:

```
C:\>
```

the working disk and directory are unimportant, or you will be in the appropriate location after following the steps we describe.

When we use the hash or pound symbol (`#`) as the prompt:

```
#
```

you will need to type in the commands as the superuser. For a Linux or Mac OS X system, this means you should log in as the system superuser by typing `su -`, or use the `sudo` keyword before the command. For a Windows system, you must be logged in with a system account that has administrator privileges.

Most of our command-line examples outside this chapter are written in a form suitable for Linux and Mac OS X; to run these instructions under Windows, simply replace the forward slash character (`/`) with the backslash character (`\`). For example, you may see an example starting the MySQL monitor program (`mysql`) from the `bin` subdirectory as follows:

```
$ bin/mysql
```

On Windows, you'd type `bin\mysql` at the Windows Command Prompt. After this chapter, we'll mostly omit the path to programs and assume that you'll call them using the appropriate path described for your installation in this chapter.

The behavior of many of the programs that we describe in this book can be modified through options. For example, you can use the `user` and `password` options to specify the username and password you want to use. Options can be specified on the command line after the program name. Some programs can also read options from a file. We explain options files in Chapter 11.

When you list options on the command line, you identify them by two adjacent hyphens:

```
$ mysql --user=saleh --password=tomcat
```


Here, we've specified the username `saleh` and the password `tomcat`.

If specified in a configuration file, the leading dashes should be omitted. For example, you would write `--user=saleh` on the command line and `user=saleh` in an options file. We'll generally omit the leading dashes in our descriptions.

Many options also have a short form that can be used only from the command line. For example, instead of writing `--user=saleh` on the command line, you can write the short form `-u saleh`. To help you understand what each command does, we consistently use the long form of each option (where one exists).

Most of the command-line utilities we describe in this book have a `help` option that you can use to discover the command syntax, including any short forms. For example, to learn about the options to use for the `mysql` program, type:

```
$ mysql --help
mysql Ver 14.12 Distrib 5.0.22, for pc-linux-gnu (i686) using readline 5.0
Copyright (C) 2002 MySQL AB
This software comes with ABSOLUTELY NO WARRANTY. This is free software,
and you are welcome to modify and redistribute it under the GPL license
Usage: mysql [OPTIONS] [database]
-?, --help Display this help and exit.
...
-p, --password[=name]
    Password to use when connecting to server. If password is
    not given it's asked from the tty.
-u, --user=name User for login if not current user.
...
```

We've shown only part of the output here. You can see that you can use the short form `-?` instead of `--help`, `-u` in place of `--user=`, and `-p` in place of `--password=`. The brackets indicate that a clause is optional; for example, you can call the `mysql` program without any command-line options or database name.

Some options assume default values if you don't specify anything. To avoid surprises, you can always explicitly specify the values you want.

When a command gets too long for the page, we show it on multiple lines, with each line ending with a backslash (`\`) symbol. For example, we might show the previous command as:

```
$ mysql \
  --user=saleh \
  --password=tomcat
```

The backslash characters indicate that this is a single command that should be typed in all on one line. You can actually type in the backslash on a Linux or Mac OS X system to continue your command on a new line, but it's not necessary.

Downloading and Verifying Files from the MySQL AB Web Site

We'll now describe in detail the steps you need to follow to get MySQL up and running on Linux, Windows, and Mac OS X systems. We'll also describe how to start, stop, and configure your MySQL server.

If you install MySQL using the packages provided by MySQL AB, you still need Apache, PHP, and Perl for the later chapters in this book. You can instead follow the instructions to install the XAMPP integrated package to get everything you need. For Linux, you can also use packages provided by your distribution.

Downloading MySQL from the MySQL AB Web Site

The MySQL AB web site usually has the very latest versions of the MySQL software. To download from this web site, follow these steps:

1. Visit the MySQL AB downloads page at <http://dev.mysql.com/downloads>. Figure 2-2 shows what this page looks like.
2. Select the MySQL version that you want. You'll normally want the latest Generally Available (GA) release; this is 5.0.67 at the time of writing. However, you can also download the cutting-edge beta version to try out new features or to help identify problems before the new version becomes the general release.
3. You'll see a long list of packages for the MySQL version you selected; Figure 2-3 shows part of this downloads page. Select the appropriate package to download for your system. In the following sections, we'll tell you what this is for each operating system and installation approach.
4. Before the file download starts, you'll probably be asked to pick a mirror server near you. Mirrors are servers that have identical copies of files for download, and are used to share the burden of many people downloading the packages. The MySQL site uses an IP-to-location database to guess where you are and will suggest some nearby servers you can download from. Selecting a mirror will start the file download.

Verifying Package Integrity with MD5

When downloading files from the Internet, it's a good idea to ensure that what you've got is what you wanted to get. For a production server, we recommend that you check the integrity of packages that you download. A simple way to do this is to compare checksums generated by a digest algorithm such as MD5.

A digest algorithm takes some data (for example, an RPM file) as input and calculates a 128-bit number, or *checksum*, from this data. With a good digest algorithm, it's practically impossible to change the data without changing the checksum, so if the checksums of two files match, you can be certain that the files are identical.

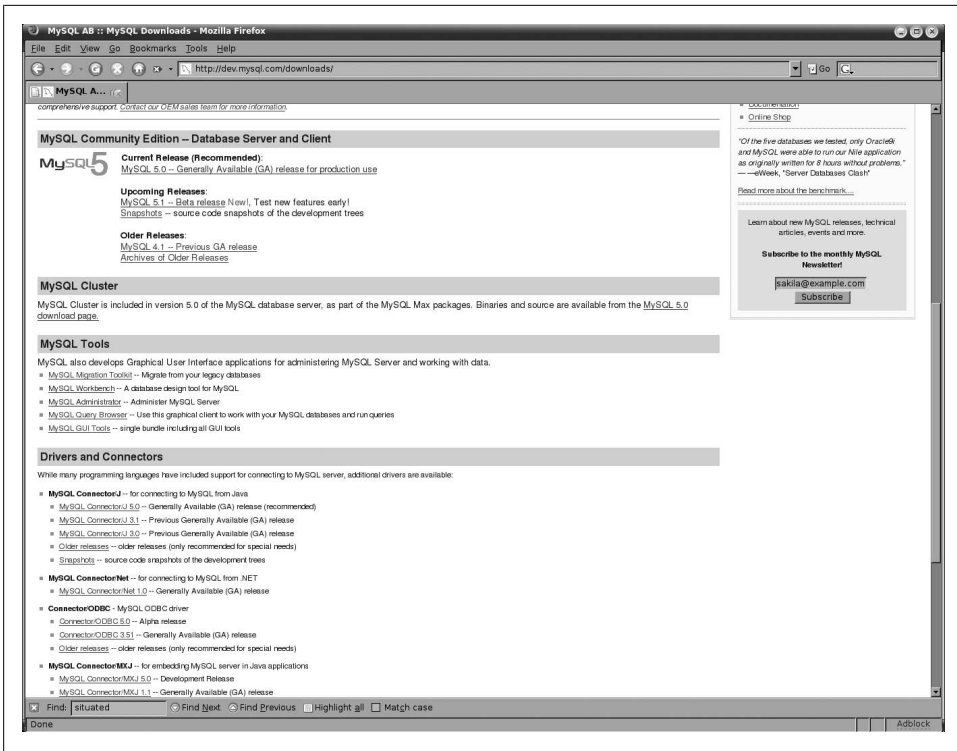


Figure 2-2. The MySQL AB downloads page

On the MySQL download page, you'll see a different string of characters such as:

MD5: 0d2a3b39e7bb4109b2f7b451b7768f34

next to each file. You should ensure that the checksum of the file you have downloaded matches the corresponding value on the downloads page.

On Linux, use the md5sum program on the downloaded file:

```
$ md5sum mysql-standard-5.0.22-linux-i686.tar.gz
0eaa7a8ec18699ce550db1713a27cda3 mysql-standard-5.0.22-linux-i686.tar.gz
```



The filename is shown in *italic* in this example because the name is likely to change, and you'll have to type in the actual name of the file you download.

On Windows, you can download and use the free winMd5Sum program from <http://www.nullriver.com/winmd5sum>. This program is very easy to use; just install and start the program, press the "... " button to browse for and select the downloaded file, and

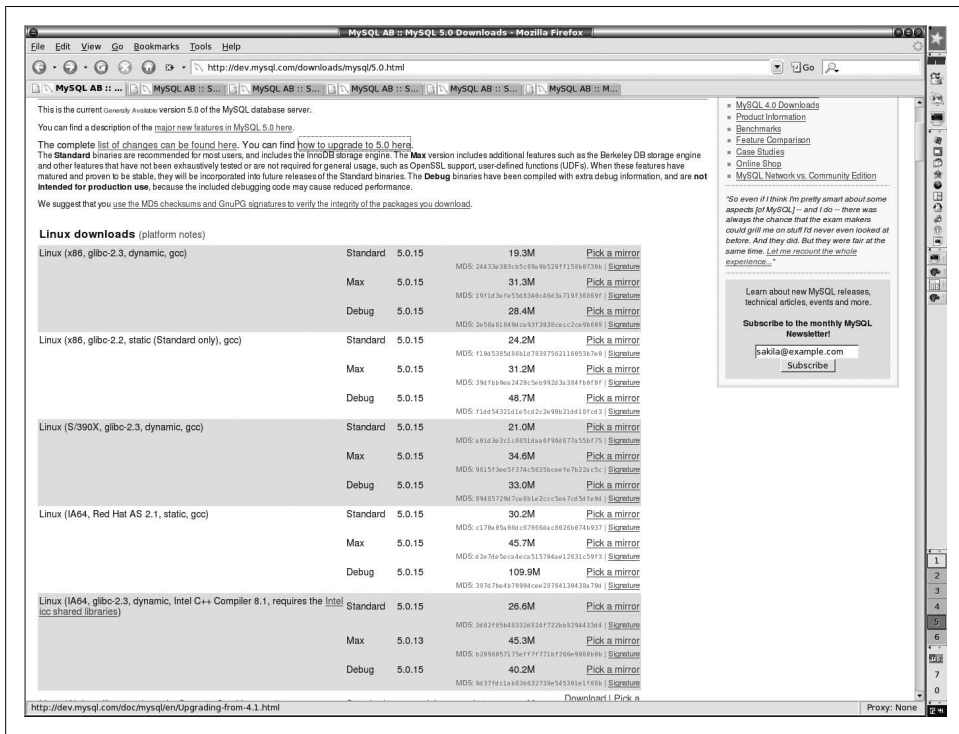


Figure 2-3. The Linux section of the MySQL downloads page

then read off the checksum value. Figure 2-4 shows what this program's dialog box looks like.

On Mac OS X, open a terminal window and use the md5 program:

```
$ md5 mysql-standard-5.0.22-osx10.4-i686.dmg
MD5
(mysql-standard-5.0.22-osx10.4-i686.dmg) =
b7d7f0878503db504e1eaed5d2518f4e
```

Digitally signed packages offer a more secure way to ensure that files have not been tampered with; however, MD5 checksums should be sufficient for most readers of this book.

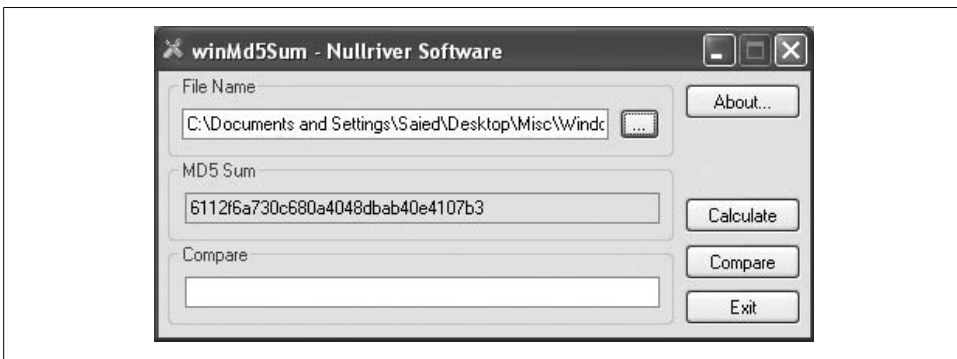


Figure 2-4. Using winMd5Sum to verify the MD5 checksum of a downloaded file



Open source projects such as MySQL, Apache, PHP, and Perl produce constantly evolving software, with new versions appearing regularly. The installation files typically include the version number in the file-name—for example, *MySQL-server-<version>.i686.rpm*. The versions of the software that you will use are almost certainly newer than the ones used in our examples, so you should substitute the appropriate version number when handling them. Of course, installation details change over time—things generally become easier—so expect some variation from the steps we discuss here. You’ll also probably find that the output we show for various programs will be slightly different from what you see on your own system.

Whenever you install software that can accept connections from other computers, you should take care to configure your computer firewall software to block connections from unauthorized systems. This is particularly important if your computer is easily accessible from the Internet, for example through your connection to your Internet Service Provider (ISP).

Installing Under Linux

There are five main ways to get MySQL up and running on a Linux system. You can:

- Install a system-wide server from packages downloaded from the MySQL AB web site. Using packages supplied by MySQL AB means that the MySQL-related files are located together in a consistent way.

MySQL AB provides these packages in the RPM format: a collection of files that can be processed and installed by the `rpm` program. The name is a vestige of the program’s origins as the Red Hat Package Manager. However, many Linux distributions other than Red Hat use RPMs for managing software installation; these include Fedora, Mandriva/Mandrake, and SUSE. The MySQL AB company also provides files for download in the format used by Debian-based distributions but

recommends that the `apt-get` method be used instead; we describe the recommended approach in this chapter.

- Install a system-wide or local server using a compressed directory (known as a gzipped tar archive) from the MySQL AB web site. This directory has all the necessary MySQL files ready to run in place; you don't need to run an installer program or place the files in a particular location on disk.
- Install a system-wide or local server by downloading the MySQL source code from the MySQL AB web site and compiling the executable programs yourself. This is the most time-consuming way of setting up Linux, but is the most flexible for power users.
- Install a system-wide server using packages created by your Linux distribution; you can download these from the Web or install them from your Linux CDs.
- Install a system-wide server by downloading the XAMPP integrated package. Note that XAMPP is not designed for use as a local server, and significant effort is required to get around this limitation.

We'll describe each of these approaches in detail. If you're not sure which approach is most suitable for you, we recommend you first try to use the packages provided by your Linux distribution.

Installing MySQL on Linux Using RPM Packages from MySQL AB

First, go to the MySQL AB downloads page following the instructions in the “Downloading MySQL from the MySQL AB Web Site” section, and scroll down the list to the part of the page with the label “Linux x86 RPM downloads.” The x86 indicates the processor type; almost all PCs today use x86 processors. If you have a more advanced type, such as an AMD 64-bit processor, you should find the appropriate part of the downloads page.

Pick RPM packages for both the MySQL server and the client, taking care that you select the correct version for your Linux distribution and your processor. These will be called something like *MySQL-server-5.0.22-0.i386.rpm* and *MySQL-client-5.0.22-0.i386.rpm*. Packages with higher CPU numbers, such as i586 or i686, are better tuned for newer machines, but won't work on older machines.

If you intend to do server benchmarking and testing, you may need to download the benchmark and test suites package (with a name like *MySQL-bench-5.0.22-0.i386.rpm*); however, you won't need them for this book.

To install the RPM files, you'll need to log in as the system root user. Open a terminal program and use the `su -` command to log in as the root user:

```
$ su -  
#
```

Change to the directory containing the MySQL RPM files you downloaded. This is typically your home directory or your desktop directory. To change to the home directory of the user `adam`, you'd type:

```
# cd ~adam
```

The location of the desktop directory depends on the Linux distribution you use, but is commonly the `Desktop` directory under the home directory. To change to the desktop directory of the user `adam`, you'd type:

```
# cd ~adam/Desktop
```

You can then install the MySQL server and MySQL client RPMs (or upgrade any existing versions) by typing:

```
# rpm --upgrade --verbose --hash \  
MySQL-server-5.0.22-0.i386.rpm MySQL-client-5.0.22-0.i386.rpm
```

If all goes well, your MySQL server should now be installed. We'll look at how to configure it in "Configuring a Newly Installed Server," later in this chapter.

Installing MySQL on Linux Using a gzipped Tar Archive from MySQL AB

Instead of using an installable package, you can download a compressed directory of the MySQL executable and support files. This process is slightly more involved than installation from a package.

Follow the instructions of "Downloading MySQL from the MySQL AB Web Site" and download the appropriate package from the "Linux (non RPM package) downloads" section of the MySQL AB downloads page. For this book, select the "standard" package, rather than the "Max" or "debug" versions.

If you're unsure what to choose, try picking the Linux download at the top of the list. This will be named something like *mysql-standard-5.0.22-linux-i686.tar.gz*.

For distribution, Linux software is often packaged using the `tar` program, and then this package is compressed using the `gzip` program, so the final file often has the file extension *.tar.gz* or *.tgz*. A *.tar* file, or its gzipped version, is often referred to as a *tarball*. You'll need to unpack, or *untar*, this package:

```
$ tar --gunzip --extract --file mysql-standard-5.0.22-linux-i686.tar.gz
```

The `gunzip` option asks the program to decompress the file first using the `gunzip` program. Some browsers automatically decompress files that have a *.gz* extension; if you get a message like "gzip: stdin: not in gzip format," this has probably happened in your case, and you can omit the `gunzip` option:

```
$ tar --extract --file mysql-standard-5.0.22-linux-i686.tar.gz
```

You should now have the directory: *mysql-standard-5.0.22-linux-i686*. To keep things simple, we'll call this the MySQL directory.

The MySQL directory is self-contained and has all the files you need to run and access the server. If you have superuser access on the Linux machine and want this MySQL server to be the system-wide instance on the machine, you should move it across to a the standard location under the */usr/local/* directory:

```
# mv mysql-standard-5.0.22-linux-i686 /usr/local/
```

and make a link */usr/local/mysql* that points to this directory:

```
# ln --symbolic /usr/local/mysql-standard-5.0.22-linux-i686 /usr/local/mysql
```

Now you can simply refer to the MySQL directory as */usr/local/mysql*. Using a symbolic link in this way allows you to have different versions of MySQL ready to run on the system, with */usr/local/mysql* pointing to the directory containing the version you want to use.

If you want to have a local installation, you can leave the MySQL directory under your home directory. You'll probably find it helpful to create the link *~/mysql* to point to the actual MySQL directory—for example:

```
$ ln --symbolic ~/mysql-standard-5.0.22-linux-i686 ~/mysql
```

With this link, you can use *~/mysql* wherever you want to refer to the *~/mysql-standard-5.0.22-linux-i686* directory.

Installing MySQL on Linux by Compiling the Source Code from MySQL AB

Given the nature of this book, we won't go into detailed compile-time settings, but will just look at how you can quickly get the server up and running.

First, you need to download the source file package from the MySQL AB downloads page, following the directions in “Downloading MySQL from the MySQL AB Web Site.” Go to the “Source downloads” section and download the “Tarball (tar.gz)” package.

After downloading, you should have a file with a name like *mysql-5.0.22.tar.gz*. Decompress this package using the following command:

```
$ tar --gunzip --extract --file mysql-5.0.22.tar.gz
```

This creates a new directory containing the MySQL source files; change your working directory to this by typing:

```
$ cd mysql-5.0.22
```

You must now compile the source code and install the resulting programs. After you've done this, you'll have a MySQL directory that has all the files you need to run and access the server. This is very similar to the tarball approach. Unlike the tarball approach, however, you need to first use the **configure** command to tell the compilation process where you want the MySQL directory to be located.

If you have superuser privileges and want your MySQL installation to be system-wide, it's best to install to a directory under the */usr/local* directory—for example, */usr/local/mysql-5.0.22*. On the other hand, if you want to run a local server, you can have the MySQL directory wherever you wish—for example, under your own home directory at *~/mysql-5.0.22*.

To install MySQL to the directory */usr/local/mysql-5.0.22*, we call the `configure` command with the target as follows:

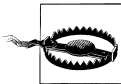
```
$ ./configure --prefix=/usr/local/mysql-5.0.22
```

If all is not well, you may see some error messages. Problems during configuration are generally due to Linux programs and libraries missing from your system; read the error messages carefully to identify the cause of the problem.

If the configuration is successful, you can use the `make` command to compile the files:

```
$ make
```

The compilation process may take a long time.



You need to use the GNU variant of the `make` program (<http://www.gnu.org/software/make>). The `make` command on most Linux systems is in fact the GNU `make` program; if you run into problems when using `make`, it might not be GNU `make`, and the problem may be resolved by using the `gmake` (GNU `make`) command instead.

When it's done, you need to install the files to the directory you specified earlier. If you've chosen to install a local server, you can simply type:

```
$ make install
```

If—as in our example—you've specified a prefix path that you can't normally write to as an ordinary user, you'll need to first log in as root:

```
$ su -
```

and then run `make install` from the root prompt to copy the compiled files to the target installation directory:

```
# make install
```

If all goes well, the MySQL files will be installed in the correct directory. You'll often find it helpful to create a link to refer to this directory easily. For example, for a system-wide server, you can make the link */usr/local/mysql* to point to the */usr/local/mysql-5.0.22* directory:

```
# ln --symbolic /usr/local/mysql-5.0.22 /usr/local/mysql
```

Now you can simply refer to the MySQL directory as */usr/local/mysql*. Similarly, if you specified the path */home/adam/mysql-5.0.22* for a local installation, you can make the link *~/mysql* to point to the *~/mysql-5.0.22* directory:

```
$ ln --symbolic ~/mysql-5.0.22 ~/mysql
```

and refer to the directory as `~/mysql`.

Again, using a symbolic link in this way allows you to configure and use different versions of MySQL on a system, with the symbolic link pointing to the directory containing the version you want to use.

Note that the configuration process assumes default values for anything that you don't specify. For example, you can explicitly set the data directory, TCP port, and socket file (more about these later):

```
$  
./configure \  
--prefix=/home/adam/mysql \  
--localstatedir=/home/adam/mysql/data \  
--with-unix-socket-path=/home/adam/mysql/mysql.sock \  
--with-tcp-port=53306
```

However, we recommend you compile only with the prefix directory specified. You can then modify other settings by passing options to MySQL from the command line; we explain how to do this in “Configuring a local server,” later in this chapter. Even better, you can specify the options in an options file as described in Chapter 11.

Installing MySQL, Apache, PHP, and Perl on Linux Using Distribution Packages

Almost all distributions include packaged versions of the main pieces of software that you need to follow this book: MySQL, the Apache web server, and support for the PHP and Perl scripting languages. In this section, we'll explain how to install these if they're not already present on your Linux system.

The three main distributions we'll cover are Red Hat, Mandriva, and Debian, as well as distributions associated with these, including Fedora, Mandrake, Ubuntu, and Knoppix. These are very widely used, and are well supported by the distributors and by the general Linux community. Configured correctly, they can automatically fetch and install the required software from the installation media or from the Internet.

Most distributions have an easy-to-use graphical package-management tool that you can use, but the command-line tools are generally more reliable, and we feel you'll better understand how things fit together by carrying out the installation from the command line.

Installation on Red Hat and Fedora Core

Red Hat is probably the most famous Linux distribution, and Fedora Core is the cutting-edge version of Red Hat's Enterprise Linux distribution. If you're installing one of these two from scratch, select the Custom installation option and, when you see the package-selection list like that shown in Figure 2-5, select (put a checkmark) next to

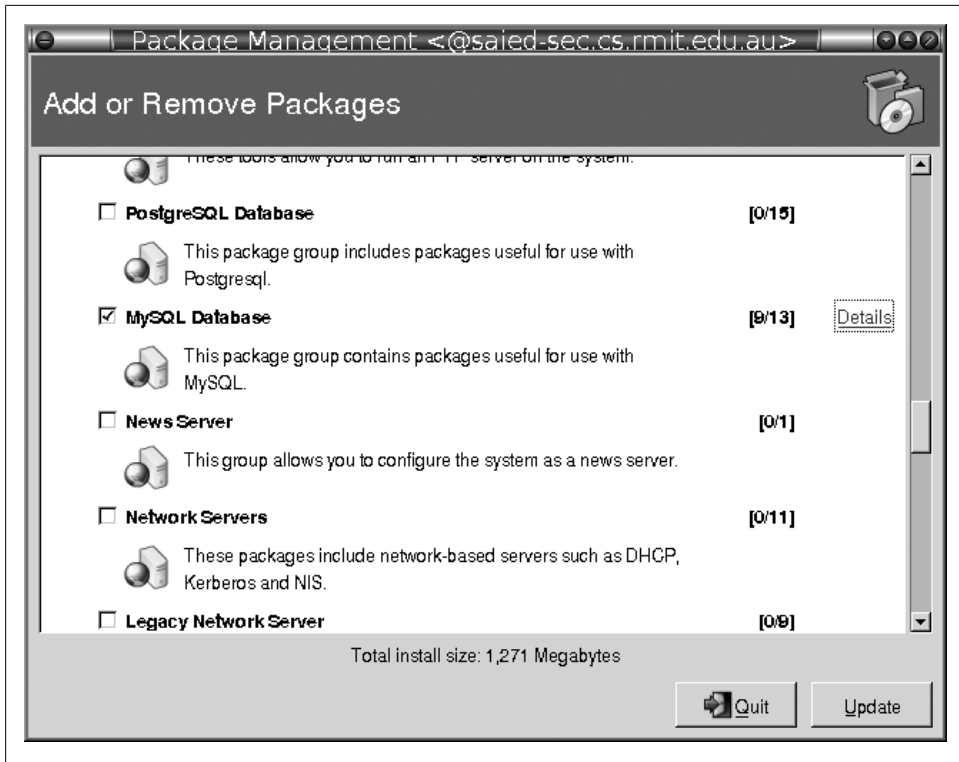


Figure 2-5. Red Hat and Fedora package options

the Web Server item. To add PHP support and PHP MySQL libraries, click on the Details link on the right and select the packages “php” and “php-mysql” from the list; you should see something similar to Figure 2-6. Once you’ve done this, return to the package-selection list and select (put a checkmark) next to the MySQL Database item. As before, click on the Details link and ensure the “php-mysql” package is selected.

If you already have a running Linux installation, you can use the `rpm` command to check whether MySQL, Apache (known as *httpd*), and PHP are already installed:

```
$ rpm --query --whatprovides mysql php php-mysql
mysql-5.0.22-1.FC5.1
mysql-server-5.0.22-1.FC5.1
httpd-2.2.0-5.1.2
php-5.1.4-1
php-mysql-5.1.4-1
```

If, as in this example, all the necessary packages are installed, you can simply skip to “Configuring a Newly Installed Server,” later in this chapter.

If the packages aren’t present, you’ll see messages like this:

```
no package provides php
```

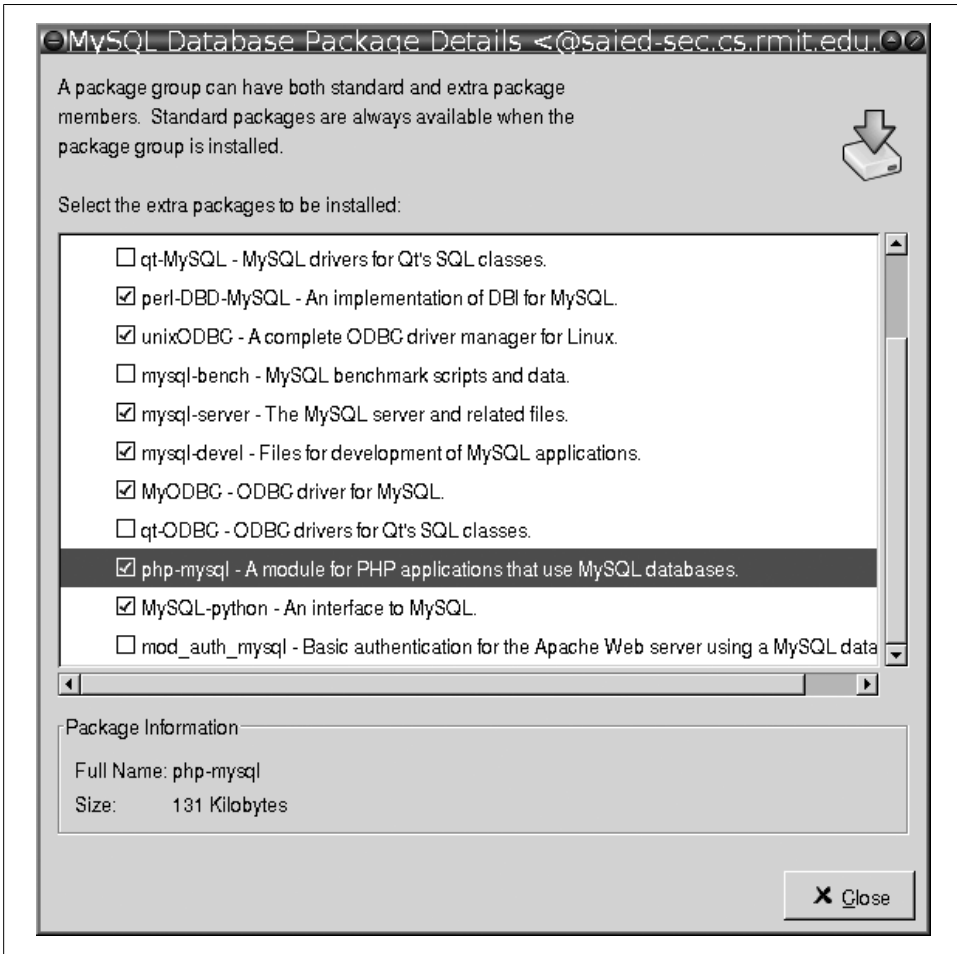


Figure 2-6. Detailed Red Hat and Fedora package options

and you'll need to install any missing packages.

Run the Package Manager by selecting Add/Remove Software from the Fedora menu. Alternatively, log in as root and type:

```
# pirut
```

You should see a window similar to the one shown in Figure 2-7. Select the List tab, and choose any of these packages that don't already have a checkmark next to them:

```
httpd-2.2.0-5.1.2.i386
```

```
Apache HTTP Server
```

```
mysql-5.0.22-1.FC5.1.i386
```

```
MySQL client programs and shared libraries
```

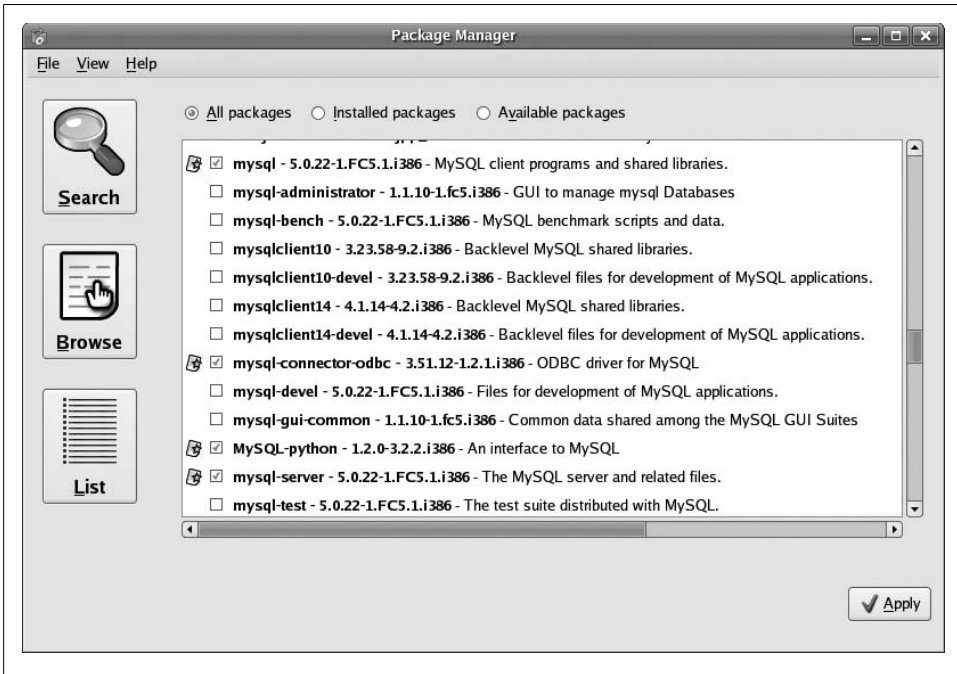


Figure 2-7. Red Hat and Fedora 5 package-management program

mysql-server-5.0.22-1.FC5.1.i386

The MySQL server and related files

php-5.1.2-5.i386

The PHP HTML-embedded scripting language (PHP Hypertext Preprocessor)

php-mysql-5.1.4-1.i386

A module for PHP applications that use MySQL databases

The version numbers you see will probably be different from the ones we've listed. Once you've selected these, click the Apply button, and the software should be installed.

If you're using an older version of Red Hat or Fedora, the easiest way to install is to log in under the root user account (by typing `su -`) and launch the package-management program shown in Figure 2-5:

```
# system-config-packages
```

Place a checkmark next to the entry for MySQL Database, and click on the Details link. You'll see a window such as that in Figure 2-6. Select the "mysql-server" and "php-mysql" packages, and then click the Close button. You'll be prompted for the Red Hat or Fedora installation CDs, and the selected packages will be installed.

If you have a relatively recent version of Red Hat or Fedora, you can also use the `yum` (short for Yellowdog Updater Modified) program to automatically download and install the necessary packages from the Internet. This is very convenient because you don't have to spend time digging up your installation CDs. More importantly, the latest version of a package generally has patches for known bugs and security vulnerabilities. If you've never used `yum` before, you need to configure it first. First, type `su -` to log in as the system root user, and then update your `/etc/yum.conf` configuration file by typing:

```
# wget http://www.fedorafaq.org/samples/yum.conf
# /bin/mv /etc/yum.conf /etc/yum.conf.bak
# /bin/mv yum.conf /etc
```

Now, update the `yum` indexes that list packages and the locations that they can be downloaded from:

```
# rpm --upgrade --verbose --hash http://www.fedorafaq.org/yum
Retrieving http://www.fedorafaq.org/yum
Preparing... ##### [100%]
1:yum-fedorafaq ##### [100%]
```

Once you've configured `yum`, you can download and install all the programs you need by simply specifying them from the command line:

```
# yum update mysql mysql-server httpd php php-mysql
[root@saiidpc ~]# yum update mysql mysql-server httpd php php-mysql
...
```

```
Could not find update match for php
Could not find update match for php-mysql
Could not find update match for mysql-server
Could not find update match for mysql
Resolving Dependencies
```

...

Package	Arch	Version	Repository	Size
Updating:				
httpd	i386	2.2.2-1.2	updates	1.1 M
Updating for dependencies:				
httpd-manual	i386	2.2.2-1.2	updates	846 k
mod_ssl	i386	1:2.2.2-1.2	updates	99 k

Transaction Summary

```
=====
Install      0 Package(s)
Update       3 Package(s)
Remove       0 Package(s)
Total download size: 2.0 M
Is this ok [y/N]: y
Downloading Packages:
(1/3): mod_ssl-2.2.2-1.2. 100% |=====| 99 kB    00:14
```

```

(2/3): httpd-2.2.2-1.2.i386 100% |=====| 1.1 MB    03:14
(3/3): httpd-manual-2.2.2 100% |=====| 846 kB    02:40
Running Transaction Test
Finished Transaction Test
Transaction Test Succeeded
Running Transaction
Updating   : httpd                      ##### [1/6]
Updating   : mod_ssl                    ##### [2/6]
Updating   : httpd-manual                ##### [3/6]
Cleanup    : mod_ssl                    ##### [4/6]
Cleanup    : httpd                      ##### [5/6]
Cleanup    : httpd-manual                ##### [6/6]

Updated: httpd.i386 0:2.2.2-1.2
Dependency Updated: httpd-manual.i386 0:2.2.2-1.2 mod_ssl.i386 1:2.2.2-1.2
Complete!

```

You'll see lots of interesting messages flash by; we haven't shown them all here. If all goes well, you should see the reassuring **Complete** status message at the end. If the latest version of a package is already installed, **yum** will tell you that it **Could not find update match for** that package. To learn more about Fedora and configuring **yum**, visit the Unofficial Fedora FAQ page (<http://www.fedorafaq.org>).

You can also download Red Hat or Fedora RPMs and install and upgrade them manually just as you would the MySQL AB ones. For example, you can visit the web site <http://rpm.phbone.net> and search for **mysql**; pick and download the RPM for Red Hat or Fedora with the highest version number. Once you've downloaded the files, log in under the root account by typing **su -**, and then install the RPM packages by typing in this command (all on one line):

```

# rpm --upgrade --verbose --hash \
  mysql-server-5.0.22-2.1.i386.rpm \
  mysql-5.0.22-2.1.i386.rpm \
  httpd-2.2.2-7.i386.rpm \
  php-5.1.4-8.1.i386.rpm

```

Installation on Mandriva

Mandriva, formerly known as Mandrake, is very easy to use for this book (we use it ourselves). MySQL, Apache, PHP, and Perl all come on the distribution CDs.

If you're installing Mandriva from scratch, choose the Expert installation option and select the MySQL server and client packages.

If you already have a running Mandriva installation, you can check whether Apache, PHP, and MySQL are already installed by typing:

```

$ rpm --query --whatprovides mysql mysql-client apache php php-mysql
MySQL-5.0.23-1mdv2007.0
MySQL-client-5.0.23-1mdv2007.0
apache-mpm-prefork-2.2.3-1mdv2007.0
apache-mod_php-5.1.4-1mdk

```

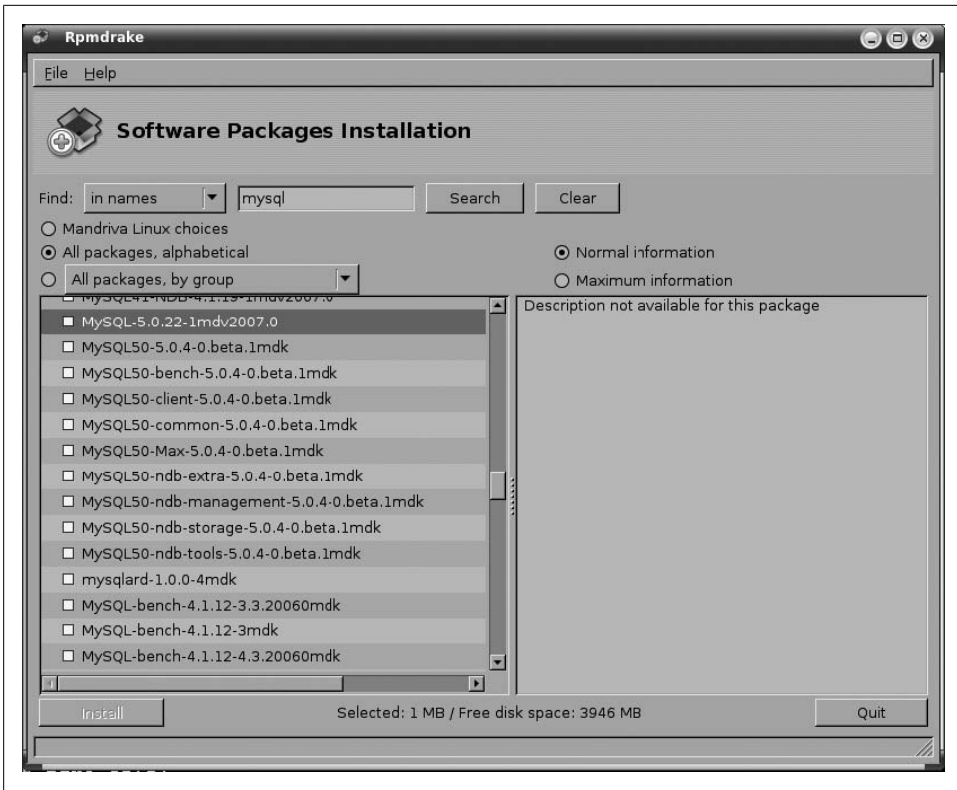


Figure 2-8. The Mandriva package-management program

```
php-cli-5.1.4-6mdv2007.0
php-mysql-5.1.4-3mdv2007.0
```

If, as in this example, all the necessary packages are installed, you can simply skip to “Configuring a Newly Installed Server,” later in this chapter.

If the packages aren’t present, you’ll see messages like this:

```
no package provides php
```

and you’ll need to install any missing packages.

The easiest way to install is to log in under the root account (by typing `su -`) and type:

```
# rpm-drake
```

This will launch the package-management program, shown in Figure 2-8. Place a checkmark next to the entries for the MySQL server and client, and click on the Install button. You’ll be prompted to insert the Mandriva installation CDs, and the selected packages will be copied and installed.

If you prefer to use the command line, you can use the `urpmi` command to specify packages to install. This will prompt you to insert the appropriate installation CDs,

and will install the packages. You may be prompted to install other related packages, depending on what's already available on your system, but in most cases, it should be painless.

If you have a fast Internet connection, you can also configure `urpmi` to download and install the very latest packages from the Internet. This is very convenient because you don't have to spend time digging up your installation CDs. More importantly, the latest version of a package generally has patches for known bugs and security vulnerabilities. To set up Internet downloads, you'll first need to tell `urpmi` where to find the packages. The easiest way to do this is to go to <http://easyurpmi.zarb.org>; this site will ask you a few questions and then provide you a list of commands you need to type in as the system root user to configure the sources (Figure 2-9 shows how this site looks.) From time to time, you should update the `urpmi` indexes by logging in as the system root user and typing:

```
# urpmi.update -a
```

Whichever approach—CDs or the Internet—you use, you just need to type `urpmi package_name` as the root user to fetch and install the required packages.

```
$ urpmi mysql mysql-client apache php php-mysql
```

```
One of the following packages is needed:
```

- 1- MySQL-5.0.23-1mdv2007.0.i586 : MySQL: a very fast and reliable SQL database engine (to install)
- 2- MySQL-Max-5.0.23-1mdv2007.0.i586 : MySQL - server with extended functionality (to install)
- 3- MySQL-NDB-4.1.12-4.3.20060mdk.i586 : MySQL - server with Berkeley DB, Innodb and NDB Cluster support (to install)

```
What is your choice? (1-3) 1
```

```
To satisfy dependencies, the following packages are going to be installed:
```

```
MySQL-5.0.23-1mdv2007.0.i586
```

```
MySQL-client-5.0.23-1mdv2007.0.i586
```

```
MySQL-common-5.0.23-1mdv2007.0.i586
```

```
apache-mod_php-5.1.4-2mdv2007.0.i586
```

```
libmysql15-5.0.23-1mdv2007.0.i586
```

```
perl-DBD-mysql-3.0006-1mdv2007.0.i586
```

```
php-mysql-5.1.4-3mdv2007.0.i586
```

```
Proceed with the installation of the 7 packages? (39 MB) (Y/n) Y
```

```
ftp://somehost.net/somedir/libmysql15-5.0.23-1mdv2007.0.i586.rpm
```

```
ftp://somehost.net/somedir/perl-DBD-mysql-3.0006-1mdv2007.0.i586.rpm
```

```
ftp://somehost.net/somedir/MySQL-common-5.0.23-1mdv2007.0.i586.rpm
```

```
ftp://somehost.net/somedir/MySQL-client-5.0.23-1mdv2007.0.i586.rpm
```

```
ftp://somehost.net/somedir/MySQL-5.0.23-1mdv2007.0.i586.rpm
```

```
ftp://somehost.net/somedir/apache-mod_php-5.1.4-2mdv2007.0.i586.rpm
```

```
installing
```

```
libmysql15-5.0.23-1mdv2007.0.i586.rpm
```

```
MySQL-client-5.0.23-1mdv2007.0.i586.rpm
```

```
MySQL-common-5.0.23-1mdv2007.0.i586.rpm
```

```
perl-DBD-mysql-3.0006-1mdv2007.0.i586.rpm
```

```
MySQL-5.0.23-1mdv2007.0.i586.rpm
```

```
apache-mod_php-5.1.4-2mdv2007.0.i586.rpm
```



Figure 2-9. The easyURPMI configuration page

```

php-mysql-5.1.4-3mdv2007.0.i586.rpm
from /var/cache/urpmi/rpms
Preparing...
 1/7: libmysql15
 2/7: MySQL-client
 3/7: perl-DBD-mysql

```

```

4/7: MySQL-common      #####...#####
5/7: MySQL              #####...#####
6/7: apache-mod_php    #####...#####
7/7: php-mysql         #####...#####

```

More information on package MySQL-5.0.23-1mdv2007.0.i586

The initscript used to start mysql has been reverted to use the one shipped by MySQL AB. This means the following changes:

- * The MYSQLD_OPTIONS="--skip-networking" option in the /etc/sysconfig/mysqld file has been removed, this is now set in the /etc/my.cnf file.
- * The MySQL Instance Manager is used by default, set use_mysqld_safe="1" in the /etc/sysconfig/mysqld file to use the old mysqld_safe script.

The extra MySQL-NDB server package has been merged into the MySQL-Max package and ndb related pieces has been split into different sub packages as done by MySQL AB. The MySQL libraries and the MySQL-common sub package uses the MySQL-Max build so that no functionality required by for example the NDB parts are lost.

The MySQL-common package now ships with a default /etc/my.cnf file that is based on the my-medium.cnf file that comes with the source code. The /etc/my.cnf file is constructed at build time of this package.

To connect to the Instance Manager you need to pass the correct command line options like in the following examples:

- * mysql -u root --password=my_password --port=2273 --protocol=TCP
- * mysql -u root --password=my_password
--socket=/var/lib/mysql/mysqlmanager.sock

Please note you also need to add a user in the /etc/mysqlmanager.passwd file and make sure the file is owned by the user under which the Instance Manager service is running under.

Here, `urpmi` has downloaded the latest versions of the programs from the Internet. During installation, some packages display messages that you should read; in our example, the MySQL package installation routine has described how the configuration has changed since older versions.

You can also download and install or upgrade the Mandriva RPMs without using `urpmi`. For example, you can visit <http://rpm.pbone.net> and search for `mysql`; pick and download the RPMs for Mandriva with the highest version number. Once you've downloaded the files, log in as the root user by typing `su -`, and then install the RPM packages by running this command (all on one line):

```

# rpm --upgrade --verbose --hash \
  MySQL-5.0.23-1mdv2007.0.i586.rpm \
  MySQL-client-5.0.23-1mdv2007.0.i586.rpm \

```

```
MySQL-common-5.0.23-1mdv2007.0.i586.rpm \
apache-mod_php-5.1.4-2mdv2007.0.i586.rpm \
libmysql15-5.0.23-1mdv2007.0.i586.rpm \
perl-DBD-mysql-3.0006-1mdv2007.0.i586.rpm \
php-mysql-5.1.4-3mdv2007.0.i586.rpm
```

Installing under Debian-based systems

Debian Linux and its derivatives use Debian *.deb* packages, rather than RPMs. The popular Ubuntu and Knoppix distributions are based on Debian.

To check whether Apache, PHP, and MySQL are already installed on a Debian-based Linux system, use the `dpkg --list` command. If any packages aren't present, the `dpkg` program will let you know:

```
$ dpkg --list mysql-common mysql-server mysql-client apache2 php5
No packages found matching mysql-client.
No packages found matching apache2.
No packages found matching php5.
Desired=Unknown/Install/Remove/Purge/Hold
| Status=Not/Installed/Config-files/Unpacked/Failed-config/Half-installed
|/ Err?=(none)/Hold/Reinst-required/X=both-problems (Status,Err: uppercase=bad)
||/ Name                Version                Description
+++-----+-----+-----+
ii  mysql-common            5.0.21-3ubuntu1        mysql database common files (e.g. /etc/mysql/my.cnf)
un  mysql-server            <none>                  (no description available)
```

On some older distributions, you may need to specify `php4` rather than `php5`.

To install MySQL, Apache, and PHP, you must first log in as the root user by typing `su -`, and then use the `apt-get install` command:

```
# apt-get --verbose-versions install mysql-common mysql-server mysql-client apache2 php5
Reading package lists... Done
Building dependency tree... Done
mysql-common is already the newest version.
The following extra packages will be installed:
apache2-common (2.0.55-4ubuntu2)
apache2-mpm-prefork (2.0.55-4ubuntu2)
apache2-utils (2.0.55-4ubuntu2)
libapache2-mod-php5 (5.1.2-1ubuntu3)
libapr0 (2.0.55-4ubuntu2)
libdbd-mysql-perl (3.0002-2build1)
libdbi-perl (1.50-1)
libnet-daemon-perl (0.38-1)
libplrpc-perl (0.2017-1)
mysql-client-5.0 (5.0.21-3ubuntu1)
mysql-server-5.0 (5.0.21-3ubuntu1)
php5-common (5.1.2-1ubuntu3)
ssl-cert (1.0.13)
Suggested packages:
apache2-doc (2.0.55-4ubuntu2)
lynx (2.8.5-2ubuntu1)
www-browser ()
php-pear (5.1.2-1ubuntu3)
```

```

dbishell ()
libcompress-zlib-perl (1.41-1)
Recommended packages:
mailx (8.1.2-0.20050715cvs-1ubuntu1)
The following NEW packages will be installed:
apache2 (2.0.55-4ubuntu2)
apache2-common (2.0.55-4ubuntu2)
apache2-mpm-prefork (2.0.55-4ubuntu2)
apache2-utils (2.0.55-4ubuntu2)
libapache2-mod-php5 (5.1.2-1ubuntu3)
libapr0 (2.0.55-4ubuntu2)
libdbd-mysql-perl (3.0002-2build1)
libdbi-perl (1.50-1)
libnet-daemon-perl (0.38-1)
libplrpc-perl (0.2017-1)
mysql-client (5.0.21-3ubuntu1)
mysql-client-5.0 (5.0.21-3ubuntu1)
mysql-server (5.0.21-3ubuntu1)
mysql-server-5.0 (5.0.21-3ubuntu1)
php5 (5.1.2-1ubuntu3)
php5-common (5.1.2-1ubuntu3)
ssl-cert (1.0.13)
0 upgraded, 17 newly installed, 0 to remove and 0 not upgraded.
Need to get 31.9MB/32.2MB of archives.
After unpacking 75.8MB of additional disk space will be used.
Do you want to continue [Y/n]? Y
...

```

The `--verbose-versions` option displays detailed information on the packages. Once you press the Y key, the required packages will be automatically downloaded and installed. We’ve left out most of the displayed messages to save space.

You can also download and install or upgrade the Debian packages without using `apt-get`; for example, you can visit the web page <http://www.debian.org/distrib/packages>, select your distribution, and search for “mysql.” Pick and download the package with the highest version number for your distribution. Once you’ve downloaded the files, log in as the root user by typing `su -`, and then install the packages by using the `dpkg --install` command—for example:

```

# dpkg --install \
  mysql-common_5.0.22-4_all.deb \
  mysql-server_5.0.22-4_all.deb \
  mysql-client-5.0_5.0.22-4_i386.deb \
  libmysqlclient15off_5.0.22-4_i386.deb

```

However, it’s quite likely that you’ll need to download other associated packages before the installation can proceed, and we recommend that you use the `apt-get` approach to automate the process. As we mentioned earlier in “Installing Under Linux,” you can also download packages in the `.deb` format from the MySQL AB downloads page.

Uninstalling MySQL

You can generally install a newer software package over an older one by using the `rpm --upgrade`, `urpmi`, `yum update`, or `apt-get install` commands described earlier. If, you actually want to remove a package altogether rather than upgrading it, you should first type `su -` to log in as the root user, and then execute the appropriate uninstall commands.

Note that the data directory that contains your database files is not actually installed but created after installation. This is typically the directory *data* under the MySQL base directory, or */var/lib/mysql* for a Linux distribution package installation. Uninstalling MySQL packages does not delete this directory, so the files containing your data should remain in place, unchanged.

For an RPM-based system such as Red Hat, Fedora, or Mandriva, use the `rpm --erase` command to uninstall specific packages. If you're unsure what the exact package names are, you can use the `rpm --query --all` command to list all the installed RPM packages, together with the `grep --ignore-case` command to show only those with "mysql" (in uppercase or lowercase letters) in their name:

```
# rpm --query --all | grep --ignore-case mysql
perl-DBD-mysql-3.0004-1mdv2007.0
MySQL-5.0.23-1mdv2007.0
libmysql15-5.0.23-1mdv2007.0
MySQL-client-5.0.23-1mdv2007.0
php-mysql-5.1.4-3mdv2007.0
MySQL-common-5.0.23-1mdv2007.0
```

Note that the *.rpm* file extension is not considered to be part of the package name. To uninstall RPM packages, you use the `rpm` command with the `--erase` option, and list the packages to remove. For example, you'd type (all on one line):

```
# rpm --erase \
  perl-DBD-mysql-3.0004-1mdv2007.0 \
  MySQL-5.0.23-1mdv2007.0 \
  libmysql15-5.0.23-1mdv2007.0 \
  MySQL-client-5.0.23-1mdv2007.0 \
  php-mysql-5.1.4-3mdv2007.0 \
  MySQL-common-5.0.23-1mdv2007.0
```

You can query and remove the packages in one go by using the `xargs` command:

```
# rpm --query --all | grep --ignore-case mysql | xargs rpm --erase
warning: /etc/my.cnf saved as /etc/my.cnf.rpm.save
#
```

On a Red Hat or Fedora system with `yum`, you can also use the `yum remove` command:

```
# yum remove mysql
[root@saiedpc yum.repos.d]# yum remove mysql
...
```

```
Dependencies Resolved
```

```

=====
Package                Arch      Version      Repository    Size
=====
Removing:
mysql                  i386      5.0.22-1.FC5.1  installed     5.5 M
Removing for dependencies:
MySQL-python          i386      1.2.0-3.2.2    installed     2.3 M
libdbi-dbd-mysql      i386      0.8.1a-1.2.1   installed      37 k
mysql-connector-odbc  i386      3.51.12-1.2.1  installed     387 k
mysql-server          i386      5.0.22-1.FC5.1  installed      22 M
perl-Class-DBI-mysql  noarch    1.00-1.fc5      installed      38 k
perl-DBD-MySQL        i386      3.0004-1.FC5    installed     324 k
php-mysql             i386      5.1.4-1         installed      176 k
=====

```

Transaction Summary

```

=====
Install      0 Package(s)
Update      0 Package(s)
Remove       8 Package(s)
Is this ok [y/N]: y
Downloading Packages:
Running Transaction Test
Finished Transaction Test
Transaction Test Succeeded
Running Transaction
  Removing : mysql-connector-odbc          #####...##### [1/8]
  Removing : perl-Class-DBI-mysql          #####...##### [2/8]
  Removing : perl-DBD-MySQL                #####...##### [3/8]
  Removing : php-mysql                     #####...##### [4/8]
  Removing : mysql                        #####...##### [5/8]
  Removing : libdbi-dbd-mysql              #####...##### [6/8]
  Removing : MySQL-python                  #####...##### [7/8]
warning: /var/log/mysqld.log saved as /var/log/mysqld.log.rpmsave
  Removing : mysql-server                  #####...##### [8/8]
=====

```

```

Removed: mysql.i386 0:5.0.22-1.FC5.1
Dependency Removed:
  MySQL-python.i386 0:1.2.0-3.2.2
  libdbi-dbd-mysql.i386 0:0.8.1a-1.2.1
  mysql-connector-odbc.i386 0:3.51.12-1.2.1
  mysql-server.i386 0:5.0.22-1.FC5.1
  perl-Class-DBI-mysql.noarch 0:1.00-1.fc5
  perl-DBD-MySQL.i386 0:3.0004-1.FC5
  php-mysql.i386 0:5.1.4-1
Complete!

```

For Debian-based systems, you can uninstall the MySQL server and client by using the `apt-get remove` command:

```

# apt-get remove mysql-server mysql-client
Reading package lists... Done
Building dependency tree... Done
The following packages will be REMOVED:
  mysql-client mysql-server
0 upgraded, 0 newly installed, 2 to remove and 1 not upgraded.
Need to get 0B of archives.

```